



Dynamic repositioning in bike-sharing systems with uncertain demand: An improved rolling horizon framework

Xiang Li ^{a,b}, Xianzhe Wang ^a, Ziyang Feng ^{a,*}

^a School of Economics and Management, Beijing University of Chemical Technology, Beijing, 100029, China

^b School of Economics and Management, Chang'an University, Xi'an, 710064, China

ARTICLE INFO

Keywords:

Bike-sharing systems
Dynamic repositioning
Uncertain demand
Improved rolling horizon approach
Heuristic algorithm

ABSTRACT

The bike-sharing system (BSS) serves as a vital shared mobility mechanism, providing first- and last-mile services in urban transportation. However, one ongoing challenge in the BSS is the spatiotemporal imbalance caused by fluctuating user demand. To address this issue, this paper studies a dynamic bike repositioning problem under scenario-based demand to achieve a balance in inventory levels for electronic fences. Specifically, a multi-period two-stage stochastic model is proposed to make the trade-off between operational costs and service quality. The model incorporates a vehicle no-return (VNR) strategy, which allows vehicles to remain at the last station at the end of each period, awaiting the next repositioning task. To solve the model, an improved rolling horizon framework is introduced to effectively handle the temporal dimension complexity of the multi-period problem. Then a customized hybrid heuristic algorithm based on parallel genetic algorithm and variable neighborhood search is developed to obtain high-quality solutions. To validate the effectiveness and applicability of the proposed method, small-scale and practical-scale numerical experiments are specifically conducted based on real-world BSS data from Beijing, China. The results demonstrate the proposed method (i) creates a substantial reduction in the number of unmet demands when multiple scenarios potentially occur compared to the determined model, (ii) yields an average reduction of 29.04% in operational costs compared to the scheme without the VNR strategy, (iii) overcomes the short-sightedness inherent in the conventional single-period rolling horizon approach, exhibiting closer to optimal performance, and (iv) is superior to baseline approaches in terms of solution quality and efficiency.

1. Introduction

The rapid proliferation of sustainable bike-sharing systems (BSS) in metropolitan cities presents an excellent solution to combat the growing urban traffic issues such as “traffic congestion” and the associated increase in “environmental pollution” [1–3]. Hundreds of worldwide major cities, such as New York, Paris, Beijing, Shanghai, etc., have already established mature BSS [4]. Statistics show that more than 23 million shared bikes are now available to individuals as a reliable one-way transportation mode for short journeys around the world [5]. However, despite the rapid development of such “semi-public” goods, operational challenges of BSS have come to light. One ongoing challenge in the BSS is the spatiotemporal imbalance of bike flows that are generated by the mobility patterns of users, as the user demand is stochastic and varies depending on the time of the day [6]. For example, during peak travel periods, some areas may experience insufficient bike supplies resulting in user dissatisfaction, while others may have excessive bike accumulation causing detrimental effects on

urban traffic. Ineffectively managed BSS designed to alleviate urban traffic congestion may become the source of traffic congestion issues that are hazards to urban management.

To address the spatiotemporal imbalance of bike flows, the platform needs to continually monitor and replenish inventory level at each electronic fence. A common approach is bike repositioning, where vehicles are dispatched from a central depot to several bike aggregation or scarcity electronic fences, typically in subway stations, business districts, or residential areas. During the repositioning process, vehicles collect excess bikes from surplus electronic fences and redistribute them to those in need, thereby maintaining a balance in inventory levels. This helps ensure the availability of bikes for users and promotes resource utilization rates while enhancing the attraction of the BSS.

Bike repositioning generally falls into two categories: static repositioning (SR) and dynamic repositioning (DR) [7]. SR usually occurs at night when there are no pick-ups or drop-offs, and the number of bikes at each electronic fence is unlikely to change. Operators strive to

* Corresponding author.

E-mail address: fengzy@buct.edu.cn (Z. Feng).

<https://doi.org/10.1016/j.omega.2024.103047>

Received 8 September 2023; Accepted 12 February 2024

Available online 21 February 2024

0305-0483/© 2024 Elsevier Ltd. All rights reserved.

balance the inventory level at each electronic fence by considering the current bike availability and the anticipated demand for the following day, and then dispatch vehicles to perform a single delivery task. However, as user demand grows in both quantity and frequency, repositioning solely at night is insufficient to match the demand throughout the day. This challenge is further exacerbated by the tidal characteristic of user demand, which peaks during morning and evening rush hours on weekdays [8]. In this case, SR is inadequate for accommodating the fluctuating demand throughout the day. Hence, addressing the inevitable inventory imbalance between user demand expectations and bike inventory levels at electronic fences during regular operating hours becomes a top priority.

The DR of shared bikes is an intricate issue that presents several complex modeling and solution challenges. First, BSS operates in a constantly changing environment associated with fluctuating demand that cannot be regarded as a fixed value on both short- and long-term timescales. So, the inherent randomness complicates the repositioning process and decreases the robustness of the system. Second, DR involves making interconnected multi-period repositioning decisions, where each decision is influenced by the previous one, creating a chain of interdependent decisions. Consequently, DR becomes a multi-period optimization problem, significantly increasing the complexity of the problem. Third, when addressing multi-period repositioning decisions, vehicles are usually assumed to depart from the depot at the beginning of each period and head back to the depot by the end of each period [9]. However, in practice, vehicles can dwell at the last fence at the end of the previous period, awaiting the subsequent task for the next period. This strategy, referred to as the vehicle no-return (VNR) strategy in this paper, eliminates the need for vehicles to travel back to the depot, especially when it is far away from the current position. In this case, determining the desired end fences for repositioning vehicles becomes essential once they have completed their repositioning tasks.

In summary, this paper aims to solve the *dynamic bike repositioning problem under stochastic demand* (DBRP-SD), adopting the VNR strategy, which contributes to the current understanding of bike repositioning challenges in urban transport systems and introduces an effective approach that promotes sustainable and efficient operational management of BBS. The main contributions can be summarized as follows.

- We present a novel approach for addressing the dynamic bike repositioning problem arising within urban transport systems, which is an innovative attempt to consider both the randomness and dynamics of user demand in the BSS. A multi-period mixed integer two-stage stochastic programming model is developed that integrates bike repositioning, vehicle routing, and VNR strategy. By linearization techniques, the proposed model is equivalently transformed into a mixed-integer linear programming model.
- An improved rolling horizon framework is applied to lessen the complexity of the multi-period model and facilitate the solving process, which can obtain closer-to-global-optimal solutions without significantly increasing time complexity compared to the conventional single-period rolling horizon approach.
- As the number of electronic fences rises, it is hard to obtain exact solutions by implicit enumeration-based methods in polynomial time. To handle the studied DBRP-SD with practical-scale cases, an efficient heuristic algorithm integrating parallel genetic algorithm and variable neighborhood search is presented.
- Finally, we apply the proposed approach to a series of small-scale instances as well as to a real-world case in Beijing, China. Potential benefits and the effectiveness of our proposed approach are discussed.

To present the research coherently, the remainder of this paper is organized as follows. Section 2 conducts a comprehensive literature review to elaborate on the state-of-the-art in the related field. Section 3 describes the DBRP-SD in detail and formulates the investigated problem into a rigorous multi-period mixed integer stochastic

programming model. To solve the model, in Section 4, three rolling horizon approaches are introduced, which consider the dynamic nature of the problem and deliver higher-accuracy solutions. Then a heuristic algorithm is discussed. Section 5 presents numerical experiments to evaluate the performance of the proposed approach. Finally, we summarize our findings and discuss the implications and opportunities for future research in Section 6.

2. Literature review

Operational-level decision-making is crucial for the management of BSS and has been the subject of extensive research over the past few decades. Researchers have conducted systematic reviews of planning and management problems in BSS ([4,10–12]). One significant challenge highlighted by these studies is the risk of supply-demand mismatching in BSS operations, which results in inefficiencies and operational issues. To address this problem, operators must make well-informed decisions regarding bike repositioning to ensure the system performance.

2.1. Operational decisions in the bike repositioning problem

In the bike repositioning problem, operational-level decisions come in a wide range of types. In general, two main strategies can be employed for bike repositioning: user-based strategy and operator-based strategy. The former aims to incentive users to take bicycles from areas with excess supply and park them in areas with insufficient demand, which is typically achieved through the implementation of financial or virtual rewards subsidy policies [13–15]. However, due to the unpredictable nature of user behavior, operators have predominantly shifted their focus towards implementing the operator-based strategy that takes a more proactive approach to managing the system. From the perspective of the BSS platform, operator-based repositioning is implemented with respect to two objectives: revenue-oriented strategies (RS) and service-oriented strategies (SS) [16]. RS primarily aims to minimize total costs or maximize total revenue, while SS primarily focuses on improving service efficiency metrics such as minimizing unmet demand or maximizing user satisfaction. In the early stages of research on the operator-based repositioning problem, station-based systems with a static approach were prevalent, with a primary focus on RS objectives, which have been considered as an extension of the pickup and delivery vehicle routing problem (PDVRP) [17]. For example, Chemla et al. [18] studied the static repositioning problem in a station-based system as a many-to-many PDVRP to minimize operational costs. The model was solved by a branch-and-cut algorithm combined with tabu search. Although the station-based system is often favored for its ease of management, reduced damage, and theft, the free-floating system stands out due to its lower construction costs and inherent flexibility in terms of user movements and parks, making it increasingly popular in the shared mobility market. However, this mode is accompanied by the potential risk of stray bikes cluttering the streets and blocking important thoroughfares, creating a significant challenge when it comes to bike repositioning. Liu et al. [19] investigated a static repositioning problem in the free-floating system and considered multiple depots, multiple visits, and multiple heterogeneous vehicles. The model took the inconvenience level into account and integrated both RS and SS. An enhanced chemical reaction optimization was developed for solving large-scale instances. Zhang et al. [20] clustered freely parked bicycles into grids and put forward three distinct mixed-integer programming models for the cooperation of “healthy and broken” bike repositioning from the perspective of RS. A hybrid heuristic algorithm of adaptive hybrid nested large neighborhood search, variable neighborhood descent, and local search was developed to solve practical-scale problems.

2.2. Dynamic and uncertain bike repositioning problem

As mentioned, the operator-based bike repositioning can be modeled using either a static or dynamic approach. The static approach is typically employed during nighttime when only a snapshot of demand is considered [21], while the dynamic approach works during the daytime combined with real-time usage of bikes and a constantly changing system, which inevitably increases the complexity of the modeling and solving process. In this regard, the underlying user demand uncertainties and dynamic decisions are two important concerns for the BSS platform [16].

User demand is particularly unpredictable and influenced by various external factors such as weather conditions, time of day, day of the week, festival, etc. However, most studies have neglected the uncertainty associated with demand and treated it as a (predictive or given) constant value at each fence as the input of the repositioning optimization model [22–24], resulting in sub-optimal system performance, inefficient resource utilization, and decreased user satisfaction. To address this issue, stochastic optimization techniques have been applied to tackle vehicle routing problems with demand uncertainty. For example, Chen et al. [16] employed a stochastic distributionally robust optimization (DRO) model, which was solved via a bisection-search approach, to address the bike repositioning problem with demand uncertainty. Fu et al. [25] formulated an integrated station location and bike repositioning problem using a two-stage robust optimization (TRO) technique to incorporate random user demand. A customized row generation was introduced to achieve high-quality solutions even under worst-case performance. Maggioni et al. [26] studied the static repositioning problem in BSS as a transshipment problem and put forward both two- and multi-stage stochastic optimization models. It can be inferred that the two-stage stochastic programming model has been widely used to facilitate decision-making while addressing uncertain parameters. However, as the number of scenarios increases, the complexity of two- and multi-stage stochastic programming models rapidly escalates, posing significant computational challenges. Therefore, a multitude of scenario-based decomposition algorithms, including L-shaped, progressive hedging, dual decomposition, etc., have been proposed to save computational time. Dell'Amico et al. [27] treated user demand as a random variable and formulated a two-stage stochastic expected value model to minimize traveling costs and unmet demand. The L-shaped method was employed to solve the problem effectively. Bashiri et al. [28] developed a two-stage stochastic programming model for the asset protection routing problem for wildfire emergencies, which was solved using Frank-Wolfe progressive hedging algorithm. Nevertheless, in the realm of bike-sharing repositioning, the sheer number of bike stations poses significant computational challenges that cannot be tackled through scenario-based decomposition algorithms. Consequently, a diverse array of heuristic algorithms, including tabu search [18], chemical reaction optimization [19], artificial bee colony algorithm [21], etc., have been employed to address the complexities of bike-sharing repositioning under uncertain demand scenarios.

Dynamic decisions have to be made for the DR problem, which often presents a multi-period optimization problem that becomes increasingly complex over time. In the literature, three main methods were commonly used for addressing the DR problem: decision support system (DSS), Markov decision process (MDP), and rolling horizon approach (RHA). The DSS is the most straightforward approach to tackling the DR problem, which converts the problem into multiple static repositioning problems by constructing a decision system to evaluate whether decisions should be made at regular intervals. Caggiani et al. [21] developed a DSS for demand statistics, which was activated at fixed gap times. This approach requires continuous monitoring and analysis of demand to generate an effective repositioning scheme for different periods among stations. From another point of view, DR can be represented as a MDP in which the BSS platform aims to develop an optimal repositioning strategy to maximize the multi-stage

reward. Legros [29] investigated a DR with a specific focus on the prioritization of bike stations. A repositioning model that minimizes the unsatisfied user demand was constructed using MDP and value iteration was employed to access optimal solutions. Luo et al. [30] adopted a MDP approach to model the DR of bikes on both sides of the road between recycling stations as a multi-stage decision problem. The approximate dynamic programming was utilized to obtain solutions with the maximum revenue. It is evident that both of the aforementioned methods possess certain limitations. The former lacks flexibility and is inefficient, while the latter introduces unnecessary assumptions, making the problem itself more complex and lacking generalization ability. A natural idea is to employ a RHA. For example, Zhang et al. [31] put forth a DR optimization method for BSS based on the multi-commodity spatiotemporal network flow and developed a mixed-integer nonlinear programming model to minimize demand dissatisfaction and bike-relocation cost. A RHA with overlapping time windows was proposed to decompose the complex problem into manageable multi-period segments. Shui and Szeto [9] considered carbon emissions and solved a green DR using RHA. To optimize large-scale instances, a heuristic method based on the artificial bee colony algorithm was designed to provide effective solutions for the repositioning problem at hand. However, one prominent issue associated with the conventional RHA is the inherent greediness in the time dimension. Due to its optimization being limited to individual sub-problem for each period, the approach has the potential to lead to shortsighted decision-making and irrational greedy actions, resulting in a decline in the overall solution quality when dealing with multi-period problems. Despite the extensive use of the RHA in solving DR problems, there still remains a lack of comprehensive studies and exploration regarding its limitations and potential improvements within the existing literature.

2.3. Summary

In Table 1, a summary of scientific publications on the bike repositioning problem in the BSS is presented according to the reposition modes (SR or DR), demand (deterministic demand (DD) or stochastic demand (SD)), objectives (RS, SS, or both), consideration of VNR strategy, models, and solution methods. Obviously, there is a noticeable dearth of literature discussing the integration of DR and SD within the BSS bike repositioning problem. Few studies consider the VNR strategy, which allows vehicles to avoid returning to the depot within each decision period. Current research lacks comprehensive modeling and discussions on the application of VNR strategies in practical repositioning problems. Furthermore, the programming model can be developed based on the different objectives of BSS platforms. Some studies prioritize minimizing operating costs by focusing solely on RS, while others prioritize minimizing unmet demand by focusing solely on SS. However, it is important to explore a model that considers both RS and SS in order to strike a balance between operating costs and service quality. To address these gaps, this paper proposes a comprehensive framework to address the dynamic bike repositioning problem, considering stochastic demand and VNR strategy in the BSS. To account for the influence of uncertain demand, a scenario-based two-stage stochastic programming model is developed using the stochastic programming (SP) method. This model is applicable to station-based, free-floating, and electronic fence systems while addressing both revenue-oriented and service-oriented objectives. To overcome the limitations of the conventional RHA, improved rolling horizon approach (IRHA) is introduced by integrating future information into the decision-making process. Moreover, a rolling heuristic algorithm using parallel computing based on a hybrid parallel genetic algorithm (PGA) and variable neighborhood search (VNS) is designed to obtain high-quality solutions for practical-scale cases, offering novel perspectives for tackling such complex optimization challenges.

Table 1
Overview of studies on the bike repositioning problem.

Literature	Mode		Demand		Objective		VNR	Models	Solution methods
	SR	DR	DD	SD	RS	SS			
Chemla et al. [18]	✓		✓		✓			MILP	Branch-and-cut tabu search
Dell'Amico et al. [17]	✓		✓		✓			MILP	Destroy and repair
Caggiani et al. [27]	✓			✓	✓	✓		SP	L-shaped
Liu et al. [19]	✓		✓		✓	✓		MILP	Chemical reaction
Du et al. [22]	✓		✓		✓			IIP	Greedy-genetic heuristic
Usama et al. [23]	✓		✓		✓	✓		MILP	Branch-and-cut
Fu et al. [25]	✓			✓	✓	✓		TRO	Row generation
Zhang et al. [20]	✓		✓		✓			MILP	Improved neighborhood search
Chen et al. [16]	✓			✓		✓		DRO	Bisection search
Zhang et al. [31]		✓	✓			✓		MILP	RHA + column generation
Caggiani et al. [21]		✓	✓		✓	✓		DSS	Spatio-temporal cluster
Shui and Szeto [9]		✓		✓		✓	✓	–	RHA + artificial bee colony
Legros [29]		✓		✓		✓		MDP	Value iteration
Chang et al. [24]		✓	✓		✓			MILP	VNS + simulated annealing
Luo et al. [30]		✓		✓	✓			MDP	Approximate dynamic programming
This paper		✓		✓	✓	✓	✓	SP	IRHA + PGA-VNS

3. Problem description and model formulation

3.1. Problem description

Let $[0, T]$ be a finite time horizon in which BSS to be operated, which can be discretized into a set of periods $\mathbb{T} = \{1, 2, \dots, t, \dots, T\}$. The DBRP-SD is modeled on a complete digraph $G(V_t, A_t)$ for each period t , where $V_t = \{0, 1, \dots, N, N+1\}$ is the set of vertices including the depot of departing and returning, vertexes 0 and $N+1$, and the subset $S_t = \{1, \dots, i, j, \dots, N\}$ represents the set of fences/stations. $A_t = \{(i, j) \mid i, j \in V_t, i \neq j\}$ is the set of arcs between each pair of distinct vertices, each is assigned non-negative traveling time c_{ij} . Since the VNR strategy is considered in this paper, a virtual vertex i' corresponding to each vertex i is introduced to represent the temporary parking stations (defined as virtual depots), where vehicles are allowed to temporarily park and then depart in the next period. Thus, three additional sets of vertices associated with each period t are particularly considered: S'_t , S_t^d , and S_t^a . $S'_t = \{1', 2', \dots, N'\}$ represents the set of virtual depots, $S_t^d = S'_t \cup \{0\}$ denotes the set of departure stations from which vehicles depart at the beginning of each period t , and $S_t^a = S'_t \cup \{N+1\}$ indicates the set of arrival stations at which vehicles arrive at the end of each period t .

A fleet of homogeneous vehicles, each with a maximum load capacity of E is associated with a dispatching cost c_i and a limited number m_i at depots (including virtual depots). A request Q_i^{tw} , which denotes the disparity between the current level of inventory and the balanced one, is given for each station $i \in S_t$ and scenario $w \in \mathbb{W}$, where $\mathbb{W}$ is the set of all possible scenarios and p^{tw} is the probability associated with scenario w , with $\sum_{w \in \mathbb{W}} p^{tw} = 1, \forall t \in \mathbb{T}$. Requests can take the form of either positive (pickup station) or negative (delivery station). The collected quantities from pickup stations can serve the delivery stations or be directed return to the depot for disposal. The repositioning task at each station is accompanied by a unit service time μ and a unit repositioning cost ρ for each bike.

In DBRP-SD, it is not always possible or practical to completely visit all stations and fulfill the requests of certain stations in some scenarios. Therefore, we model DBRP-SD using a two-stage stochastic paradigm, where a recourse function allows demands to be partially fulfilled at the cost of a penalty at the second stage [27]. When a demand is violated, the unsatisfied task is penalized by incurring a penalty δ for each repositioning demand of slack and surplus h_i^{tw} at each station $i \in S_t$, which is then accumulated for the next period. The penalty is chosen to be a fraction of the vehicle dispatch and traveling cost, which is calibrated in Section 5.2.

To provide a clear exposition of DBRP-SD, we next elaborate on the operational mechanism of the dynamic bike repositioning problem that considers multiple scenarios, multiple periods, and VNR strategy. As

illustrated in Fig. 1, we consider a complete digraph that involves one depot and five stations. The horizon is discretized into three periods. At the beginning of the first period, two vehicles are positioned at depot 0 (where 6 represents the depot where vehicles return), with a maximal capacity of 10. The repositioning demands (i.e., the difference between inflow and outflow of orders) at stations within three periods under two scenarios are presented in Table 2. In Fig. 1, the number of loaded/unloaded bikes associated with each station is composed of a positive/negative number and a positive subscript enclosed in parentheses. The former represents the fulfilled demand (positive indicating loading and negative indicating unloading), while the latter indicates the corresponding scenario. For example, $(+5)_1$ at station 1 period 1 represents that 5 bikes are loaded onto the vehicle under scenario 1. Three costs are considered as key metrics: C_1 represents traveling time cost, with a unit coefficient $\eta = 1$. For simplicity, we assume a constant travel time of 1 on the connected arcs shown in Fig. 1. C_2 denotes the vehicle dispatch cost from the depot, with a unit coefficient $c_i = 10$ for $i \in \{0\}$ and $c_i = 2$ for virtual depots $i \in S'_t$. C_3 indicates the penalty cost for unmet demand, with a unit penalty coefficient $\delta = 5$.

When decisions are made based on known and determined demand, specifically considering scenario 1, the bike repositioning plan is presented in Fig. 1(a). In the first period, one vehicle is dispatched from the depot and visits stations 1, 2, 3, 4, and 5 in sequence. At the end of the first period, the vehicle dwells at station 5 (i.e., virtual depot represented by gray lined circles) with 6 bikes on board, waiting for the task in the second period. In this case, we have $S_1^d = \{0\}$ and $S_1^a = S'_1 \cup \{6\}$. Moving on to period 2, two vehicles are dispatched: one from the virtual station $5'$ carrying 6+3 bikes and the other from the depot 0 carrying 9 bikes. Here, we have $S_2^d = \{5', 0\}$ and $S_2^a = S'_2$. At the end of the second period, two vehicles dwell at stations 2 and 3 with 1 and 4 bikes on board respectively. In the final period, all vehicles head to the depot 6, $S_3^d = \{2', 3', 0\}$ and $S_3^a = \{6\}$. The associated operational costs C_1 , C_2 , and C_3 are, respectively, 14, 26, and 0 (all repositioning demands are satisfied) for the three periods. The number of bikes carried out from the depot is equal to the number of bikes carried back to the depot.

However, relying solely on a single determined scenario often yields overly restrictive solutions that may fail to meet the practical-scenario repositioning demands, incurring penalty costs and unsatisfactory services. This limitation stems from the inherent uncertainty of user demand, which introduces multiple potentialities. For example, assuming that scenario 1 has changed to scenario 2, as depicted in Fig. 1(b), the repositioning demand exhibits a significant variation at station 4 within the second period, resulting in a substantial amount of unmet demand (highlighted in bold and red). This unmet demand triggers a “domino effect” in the subsequent period, affecting both stations 2, 3, and 4, which also experience unfulfilled demand. The associated

Table 2
Repositioning demands at different stations under two scenarios and three periods.

	Station 1		Station 2		Station 3		Station 4		Station 5	
	$w = 1$	$w = 2$	$w = 1$	$w = 2$	$w = 1$	$w = 2$	$w = 1$	$w = 2$	$w = 1$	$w = 2$
$t = 1$	5	5	3	3	-7	-7	5	6	-2	-2
$t = 2$	-1	-4	-7	-6	4	6	-9	-4	3	3
$t = 3$	4	5	-1	-5	3	5	1	1	-1	-6

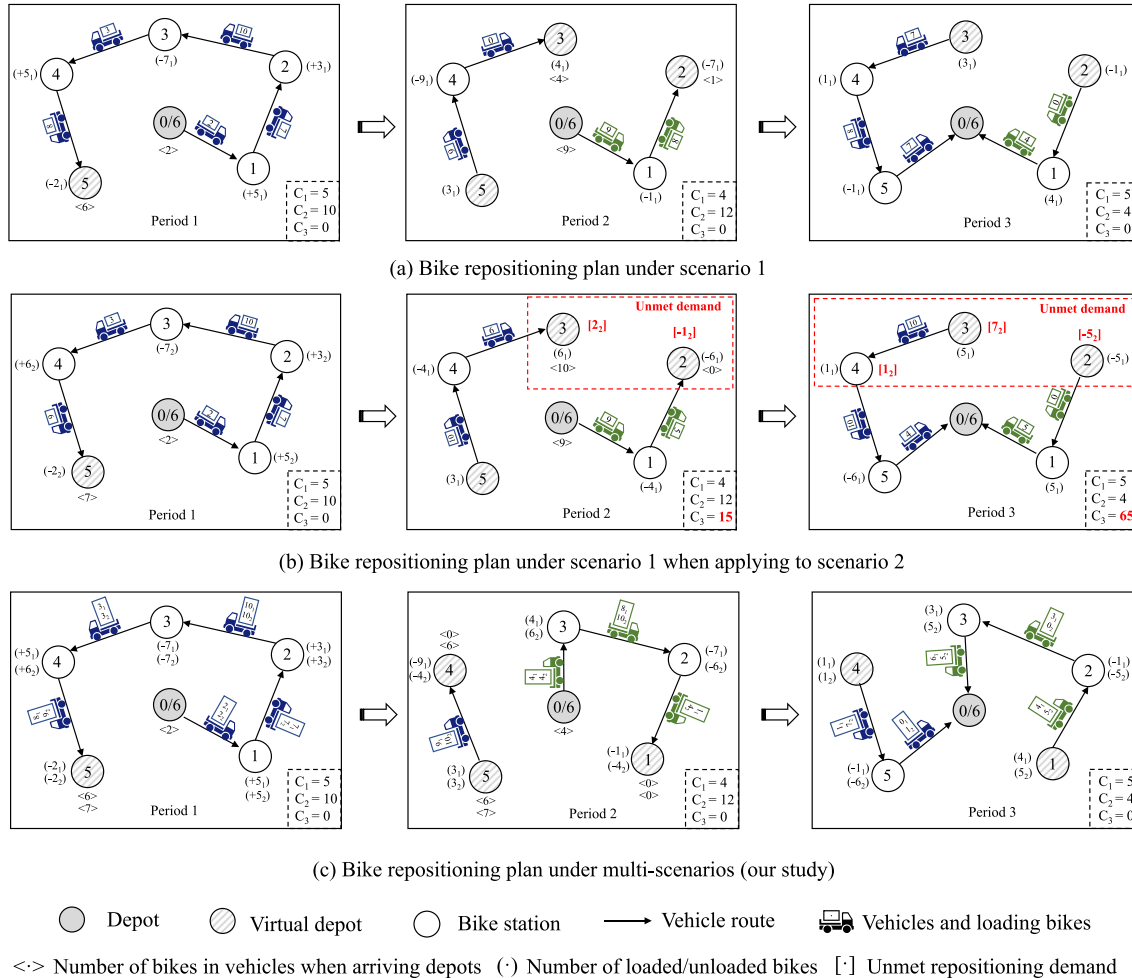


Fig. 1. Dynamic bike repositioning process diagram. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

operational costs C_1 and C_2 remain unchanged, while the penalty cost C_3 increases to 80 for the three periods. Clearly, relying on deterministic demand assumption reduces the system robustness. Therefore, it becomes necessary to consider multiple demand scenarios. As presented in Fig. 1(c), two scenarios are incorporated in the optimization model with the same probability. The repositioning plan for period 1 remains the same as in the previous two situations, where a vehicle carries 2 bikes from the depot and visits stations 1–5 sequentially, then dwells at station 5. At the end of the second period, the vehicle is temporarily parked at station 4, which experiences high demand variation, rather than station 3. Meanwhile, another vehicle carrying 4 bikes is dispatched from the depot to visit stations 3, 2, and 1 in order. In the last period, all vehicles return to the depot. The associated operational costs C_1 , C_2 , and C_3 are the same as the repositioning plan when considering only scenario 1, without incurring any unmet demand. It is evident that by incorporating multiple scenarios in the optimization process, the system can proactively address the unmet demand and mitigate its negative consequences.

In summary, we address a dynamic bike repositioning problem: Given the number of periods, station coordinates, vehicle quantity and

capacity, and repositioning demand scenarios with corresponding probabilities. The goal is to minimize the operational costs while ensuring service quality to provide a repositioning plan including vehicle routes, the initial number of bikes carried from the depot, and the load/unload quantity of bikes at each station across various periods. Consequently, the detailed scenario-based mathematical model is introduced below.

3.2. Model formulation

In this part, we present a mathematical model for the considered DBRP-SD. To account for the uncertainty in demand across different scenarios, a two-stage stochastic expected value model is developed to incorporate discrete random demands into the formulation. The first-stage model makes decisions before the realization of the random variable, while the second-stage model makes decisions after the realization of the random variable. Specifically, in the case of discrete random variables, the optimal value of the second-stage model is weighted and averaged into the objective value of the first-stage model to help obtain the two-stage stochastic expected value model. This

Table 3
Sets, parameters, and variables of the model.

Notations	Definition
$\mathbb{T}$	Set of periods
$\mathbb{W}_t$	Set of scenarios in period t
S_t	Set of bike stations in period t
S'_t	Set of virtual depots in period t
S_t^d	Set of departure stations in period t
S_t^a	Set of arrival stations in period t
V_t	Set of bike stations and depots in period t
A_t	Set of arcs in period t
Parameters	
ς_{ij}	Traveling time between stations i and j
Q_i^w	Repositioning demand at station i in period t and scenario w
p^w	Probability of scenario w occurring in period t
m_i^t	Number of available vehicles at depot i in period t
L_i^t	Earliest service time at station i in period t
R_i^t	Latest service time at station i in period t
c_i	Unit dispatch cost from depots i
μ	Unit bike repositioning service time
ρ	Unit bike repositioning cost
δ	Unit penalty cost for unmet demand
η	Unit traveling time cost
E	Maximum capacity of repositioning vehicles
First-stage variables	
x_{ij}^t	0–1 binary variable, if a vehicle moves to station j after visiting station i , $x_{ij}^t = 1$; otherwise, $x_{ij}^t = 0$, in period t
z_i^t	0–1 binary variable, if a vehicle visits station i , $z_i^t = 1$; otherwise, $z_i^t = 0$, in period t
τ_i^t	Integer variable, the vehicles' arrival time at station i in period t
Second-stage variables	
y_i^w	Integer variable, the available capacity of vehicle after it visits station i in period t , scenario w
q_i^w	Integer variable, the quantity of bike loaded (if positive) or unloaded (if negative) on the repositioning vehicle at station i in period t , scenario w
h_i^w	Auxiliary integer variable, the number of unmet repositioning demand at station i , in period t , scenario w
a_i^t	Auxiliary integer variable, the available vehicle capacity from the depot

modeling paradigm aligns well with the context of the dynamic bike repositioning problem, where the delivery route needs to be determined before the demand is realized, and the load/unload quantity can be optimized after the demand is realized. Prior to the modeling process, we establish several assumptions that will be maintained throughout this paper:

Assumption 1. There is a single depot located in the operational zone, which contains an abundance of bikes.

Assumption 2. The repositioning vehicles, which are completely homogeneous in terms of capacity, speed, start-up cost, etc., are dispatched uniformly from the depot.

Assumption 3. Each station can be served only once by one repositioning vehicle during each period. If an electronic fence requires the repositioning of more bicycles than the vehicle's capacity, it can be divided into multiple electronic fences.

Assumption 4. The retrieval of damaged bikes is not considered in the bike repositioning process.

Assumption 1 is reasonable as bike-sharing systems are often operated within distinct regions, and each is managed by a single depot. This assumption is consistent with prior studies such as Erdoğan et al. [32], Shui and Szeto [9], and Zhang et al. [20]. Assumption 2 holds because operators tend to standardize the vehicles since employing homogeneous vehicles brings the benefits of unified management and the potential reduction of the complexity in solving, which has been widely accepted in Dell'Amico et al. [6], Zhang et al. [31], Al Chami Z et al. [33] and Chang et al. [24]. Assumption 3 is a standard consideration for vehicle routing problems. It is not economical for the

bike-sharing platform to dispatch multiple vehicles to a single station repetitively. If a single station's repositioning demand surpasses the vehicle's capacity, it can be segmented into multiple stations, allowing the model to seamlessly handle multiple visits to the same station within this assumption. Similar assumptions are adopted by Ho and Szeto [34], Zhang et al. [31], Al Chami Z et al. [33] and Fu et al. [25]. Besides, dynamic repositioning occurs frequently within a day, whereas damaged bike collection is much less frequent, potentially taking place in several days. Therefore, Assumption 4 consciously sets aside the less common damaged bike collection to emphasize the focus of this study, which aligns with the approach taken by Liu et al. [19], Legros [29], and Luo et al. [30].

Some closely related sets, parameters, and variables are listed in Table 3.

3.2.1. First-stage problem

The first stage aims to determine the strategic decisions regarding the visited stations, traveling routes of repositioning vehicles, and their arrival times at stations prior to the realization of uncertain demands. From this, the goal of the first stage is to minimize the total costs, encompassing the vehicle traveling cost, the vehicle dispatching expense, and the recourse delivery and penalty cost of the second stage, where $\Gamma_t(\mathbf{x}, \mathbf{z}, \boldsymbol{\tau}, \xi(w))$ represents the second-stage objective value corresponding to the realization of the random scenario $\xi(w)$ and $\mathbb{E}$ denotes the expected value operator. The objective function of the first stage problem can be formulated as:

$$\min C = \sum_{t \in \mathbb{T}} \left[\eta \sum_{(i,j) \in A_t} \varsigma_{ij} x_{ij}^t + \sum_{i \in S_t^d} c_i \sum_{j \in S_t} x_{ij}^t + \mathbb{E} \left[\Gamma_t(\mathbf{x}, \mathbf{z}, \boldsymbol{\tau}, \xi(w)) \right] \right]. \quad (1)$$

Next, we give a comprehensive explanation of the constraints associated with the first stage model within each period (intra-period

constraints). Constraint (2) imposes an upper limit on the number of vehicles allowed to dispatch from depots $i \in S_t^d$ at the beginning of period t . To be precise, multiple vehicles can depart from the depot, whereas only one vehicle is permitted to depart from virtual depots, in accordance with Assumption 3. Constraint (3) guarantees vehicles must arrive at their designated destination stations by the end of period t . Constraint (4) denotes that a station $i \in S_t$ is visited if a vehicle arrives at it from any other stations $j \in S_t \cup S_t^d$, ensuring that each station can be served only once except for the depot. Flow conservation at each station is preserved by constraints (5)–(6), where the former restricts the number of vehicles remains constant upon inflow and outflow at station $i \in S_t$, while the later imposes that the total number of vehicles remains consistent when departing from departure stations $i \in S_t^d$ and arriving at arrival stations $i \in S_t^a$ within each period t . Eq. (7) is a service time related constraint, ensuring the schedule feasibility of vehicles with respect to the corresponding period when visiting stations.

$$\sum_{j \in S_t} x_{ij}^t \leq m_i^t, \quad \forall i \in S_t^d, t \in \mathbb{T}, \quad (2)$$

$$\sum_{i \in S_t} \sum_{j \in S_t^a} x_{ij}^t \geq 1, \quad \forall t \in \mathbb{T}, \quad (3)$$

$$\sum_{j \in S_t \cup S_t^d, j \neq i} x_{ji}^t = z_i^t, \quad \forall i \in S_t, t \in \mathbb{T}, \quad (4)$$

$$\sum_{j \in S_t \cup S_t^a, j \neq i} x_{ij}^t = \sum_{j \in S_t \cup S_t^d, j \neq i} x_{ji}^t, \quad \forall i \in S_t, t \in \mathbb{T}, \quad (5)$$

$$\sum_{i \in S_t^d} \sum_{j \in S_t} x_{ij}^t = \sum_{i \in S_t^a} \sum_{j \in S_t} x_{ji}^t, \quad \forall t \in \mathbb{T}, \quad (6)$$

$$L_i^t \leq \tau_i^t \leq R_i^t, \quad \forall i \in V_t, t \in \mathbb{T}. \quad (7)$$

Among different periods, constraint (8) (inter-period constraint) has been considered to impose the number of vehicle visits at each station aligns within each adjacent period, indicating that if a vehicle remains at a specific virtual depot j corresponding to a station i at the end of period t , it will serve the station i first at the beginning of the period $t+1$.

$$x_{ij}^{t-1} = x_{ji}^t, \quad \forall i \in S_t, j \in S_t^d, t \in \mathbb{T}, t > 1. \quad (8)$$

Constraints (9)–(11) define the variable domains, where $\mathbb{Z}^+$ indicates the set of non-negative integers.

$$x_{ij}^t \in \{0, 1\}, \quad \forall i, j \in V_t \cup S_t^d, i \neq j, t \in \mathbb{T}, \quad (9)$$

$$z_i^t \in \{0, 1\}, \quad \forall i \in S_t, t \in \mathbb{T}, \quad (10)$$

$$\tau_i^t \in \mathbb{Z}^+, \quad \forall i \in V_t, t \in \mathbb{T}. \quad (11)$$

3.2.2. Second-stage problem

Based on the visited stations, service routes, and arrival times of repositioning vehicles in the first stage, the goal of the second stage is to determine the transport capacity and service quantities under different scenarios. Hence, given the first-stage decision vector $(\mathbf{x}, \mathbf{z}, \boldsymbol{\tau})$ and denoting the realization of the random data vector $\xi(w)$, the second-stage problem under a particular scenario $w, w \in \mathbb{W}$, defined as $\Gamma_t(\mathbf{x}, \mathbf{z}, \boldsymbol{\tau}, \xi(w))$, is expressed as follows:

$$\Gamma_t(\mathbf{x}, \mathbf{z}, \boldsymbol{\tau}, \xi(w)) = \min \sum_{i \in S_t} (\rho |q_i^{tw}| + \delta |h_i^{tw}|). \quad (12)$$

The first term of function (12) is the bike repositioning cost that can be regarded as the staff service cost, while the second term is the penalty cost for unmet demand. Next are the intra-period constraints that within each period:

$$y_i^{tw} \geq y_j^{tw} + q_j^{tw} - M(1 - x_{ij}^t), \quad \forall i, j \in S_t, i \neq j, t \in \mathbb{T}, \quad (13)$$

$$y_j^{tw} \geq y_i^{tw} - q_i^{tw} - M(1 - x_{ij}^t), \quad \forall i, j \in S_t, i \neq j, t \in \mathbb{T}, \quad (14)$$

$$\max\{0, -q_i^{tw}\} \leq y_i^{tw} \leq \min\{E, E - q_i^{tw}\}, \quad \forall i \in S_t, t \in \mathbb{T}, \quad (15)$$

$$y_i^{tw} + q_i^{tw} - a_i^t \leq M(1 - x_{0i}^t), \quad \forall i \in S_t, t \in \mathbb{T}, \quad (16)$$

$$y_i^{tw} + q_i^{tw} - a_i^t \geq M(x_{0i}^t - 1), \quad \forall i \in S_t, t \in \mathbb{T}, \quad (17)$$

$$|q_i^{tw}| \leq Ez_i^t, \quad \forall i \in S_t, t \in \mathbb{T}, \quad (18)$$

$$\tau_i^t + \varsigma_{ij} + \mu |q_i^{tw}| - M(1 - x_{ij}^t) \leq \tau_j^t, \quad \forall i \in S_t \cup \{0\}, j \in S_t \cup \{n+1\}, i \neq j, t \in \mathbb{T}. \quad (19)$$

Constraints (13)–(14) impose that, if $x_{ij}^t = 1$, then $y_j^{tw} = y_i^{tw} - q_j^{tw}$, ensuring the available capacity of a vehicle at station j before departure is equal to the difference between the available capacity of the vehicle at station i after servicing and the quantity of bikes loaded (or unloaded) at station j . Constraint (15) keeps the upper and lower bounds of the available capacity for repositioning vehicles. Constraints (16)–(17) unify the initial number of bikes on vehicles across all scenarios when departing from the depot. Since stations are allowed not to be visited albeit with a penalty cost, constraint (18) ensures that bikes are loaded (or unloaded) only when the station is visited. Constraint (19) represents the service time at each station.

Among different periods, the inter-period constraints of the second-stage problem including:

$$y_i^{tw} \leq y_i^{(t-1)w} - q_i^{tw} + M(1 - \sum_{j \in S_t^d} x_{ji}^t), \quad \forall i \in S_t, t \in \mathbb{T}, t > 1, \quad (20)$$

$$y_i^{tw} \geq y_i^{(t-1)w} - q_i^{tw} - M(1 - \sum_{j \in S_t^d} x_{ji}^t), \quad \forall i \in S_t, t \in \mathbb{T}, t > 1, \quad (21)$$

$$h_i^{tw} = Q_i^{tw} + h_i^{(t-1)w} - q_i^{tw}, \quad \forall i \in S_t, t \in \mathbb{T}, t > 1, \quad (22)$$

$$\min\{0, Q_i^{tw} + h_i^{(t-1)w}\} \leq q_i^{tw} \leq \max\{0, Q_i^{tw} + h_i^{(t-1)w}\}, \quad \forall i \in S_t, t \in \mathbb{T}, t > 1. \quad (23)$$

Constraints (20)–(21) impose that the available capacity of vehicles when leaving from and arriving at virtual depots in current and previous periods are matched. Constraint (22) calculates the unmet demand at each station, which equals the repositioning demand Q_i^{tw} minus the number of served bikes q_i^{tw} , plus the unmet demand from the previous period $h_i^{(t-1)w}$. Constraint (23) restricts that bikes are loaded (or unloaded) in the current period to not exceed the sum of demand in the current period and unmet demand in the previous period.

Additionally, constraint (24) initializes the unmet repositioning demand in the first period. Constraints (25)–(26) specify the domain of the decision variables, where $\mathbb{Z}$ represents the set of integers.

$$h_i^0 = 0, \quad \forall i \in S_t, \quad (24)$$

$$y_i^{tw} \in \mathbb{Z}^+, \quad \forall i \in S_t, t \in \mathbb{T}, \quad (25)$$

$$q_i^{tw}, h_i^{tw} \in \mathbb{Z}, \quad \forall i \in S_t, t \in \mathbb{T}. \quad (26)$$

3.2.3. Optimization model and linearization processing

Assuming that the uncertain demand has a finite number of possible realized values, the two-stage stochastic model for the considered problem can be reformulated into a computationally tractable form:

$$\begin{cases} \min & \sum_{t \in \mathbb{T}} \left[\eta \sum_{(i,j) \in A_t} \zeta_{ij} x_{ij}^t + \sum_{i \in S_t^d} c_i \sum_{j \in S_t} x_{ij}^t + \sum_{w \in \mathbb{W}_t} p^{tw} \sum_{i \in S_t} \left(\rho |q_i^{tw}| + \delta |h_i^{tw}| \right) \right] \\ \text{s.t.} & \text{constraints (2)–(11)} \\ & \text{where, for all } w, \\ & \text{constraints (13)–(26)} \end{cases} \quad (27)$$

where p^{tw} represents the probability of scenario w in period t . The optimization model (27) consists of integer and binary variables, with a nonlinear objective function and constraints. The presence of nonlinear components, including absolute value, maximum value, and minimum value functions within the second-stage objective function (12) and constraints (15), (18)–(19), and (23). To ensure efficient solvability, it is necessary to transform the model (27) into a mixed integer linear programming (MILP) problem. Here we employ linearization techniques to linearize the nonlinear objective functions and constraints. Some extra auxiliary variables as well as lemmas are introduced as follows. Note that the following lemmas, although not proved in this paper, can be easily verified.

Lemma 1. For the absolute terms $|q_i^{tw}|$ and $|h_i^{tw}|$ in the model (27), two auxiliary variables, $(q_i^{tw})^+$ and $(h_i^{tw})^+$, are defined as $(q_i^{tw})^+ = |q_i^{tw}|$ and $(h_i^{tw})^+ = |h_i^{tw}|$ respectively. The following auxiliary constraints should be considered:

$$\begin{cases} (q_i^{tw})^+ \geq q_i^{tw}, \\ (q_i^{tw})^+ \geq -q_i^{tw}, \\ (h_i^{tw})^+ \geq h_i^{tw}, \\ (h_i^{tw})^+ \geq -h_i^{tw}, \\ (q_i^{tw})^+, (h_i^{tw})^+ \in \mathbb{Z}^+, \end{cases} \quad \forall i \in S_t, t \in \mathbb{T}. \quad (28)$$

Based on Lemma 1, the nonlinear objective function (12) and constraints (18)–(19), which contains absolute value functions can be equivalently replaced by the following linear formulations (29) and (30)–(31) respectively, with the consideration of constraint (28).

$$\Gamma_t(\mathbf{x}, \mathbf{z}, \boldsymbol{\tau}, \xi(w)) = \min \sum_{i \in S_t} (\rho (q_i^{tw})^+ + \delta (h_i^{tw})^+), \quad (29)$$

$$(q_i^{tw})^+ \leq E z_i^t, \quad \forall i \in S_t, t \in \mathbb{T}, \quad (30)$$

$$\tau_i^t + t_{ij} + \mu (q_i^{tw})^+ - M(1 - x_{ij}^t) \leq \tau_j^t, \quad \forall i \in S_t \cup \{0\}, j \in S_t \cup \{n+1\}, i \neq j, t \in \mathbb{T}. \quad (31)$$

Lemma 2. For $i \in S_t$ and $t \in \mathbb{T}$, the nonlinear constraint (15), can be equivalently replaced by the following linear constraints:

$$\begin{cases} y_i^{tw} \geq 0, \\ y_i^{tw} \geq -q_i^{tw}, \\ y_i^{tw} \leq E, \\ y_i^{tw} \leq E - q_i^{tw}, \end{cases} \quad \forall i \in S_t, t \in \mathbb{T}. \quad (32)$$

Lemma 3. For constraint (23), four auxiliary binary variables β_i^{tw} , γ_i^{tw} , ζ_i^{tw} , ϑ_i^{tw} are introduced to denote the relationship between 0 and $Q_i^{tw} + h_i^{(t-1)w}$. I.e., for the min term of constraint (23), when $\beta_i^{tw} = 1$, and $\gamma_i^{tw} = 0$, $Q_i^{tw} + h_i^{(t-1)w} > 0$; when $\beta_i^{tw} = 0$, and $\gamma_i^{tw} = 1$, $Q_i^{tw} + h_i^{(t-1)w} < 0$; when $\beta_i^{tw} = 1$, and $\gamma_i^{tw} = 1$, $Q_i^{tw} + h_i^{(t-1)w} = 0$. Similarly, for the max term of constraint (23), when $\zeta_i^{tw} = 1$, and $\vartheta_i^{tw} = 0$, $Q_i^{tw} + h_i^{(t-1)w} < 0$; when

$\zeta_i^{tw} = 0$, and $\vartheta_i^{tw} = 1$, $Q_i^{tw} + h_i^{(t-1)w} > 0$; when $\zeta_i^{tw} = 1$, and $\vartheta_i^{tw} = 1$, $Q_i^{tw} + h_i^{(t-1)w} = 0$. By introducing a large positive value M , the constraint (23) is equivalently replaced by the following linearized constraints:

$$\begin{cases} q_i^{tw} \geq -M(1 - \beta_i^{tw}), \\ q_i^{tw} \geq Q_i^{tw} + h_i^{(t-1)w} - M(1 - \gamma_i^{tw}), \\ q_i^{tw} \leq M(1 - \zeta_i^{tw}), \\ q_i^{tw} \leq Q_i^{tw} + h_i^{(t-1)w} + M(1 - \vartheta_i^{tw}), \quad \forall i \in S_t, t \in \mathbb{T}, t > 1. \\ \beta_i^{tw} + \gamma_i^{tw} \geq 1, \\ \zeta_i^{tw} + \vartheta_i^{tw} \geq 1, \\ \beta_i^{tw}, \gamma_i^{tw}, \zeta_i^{tw}, \vartheta_i^{tw} \in \{0, 1\}, \end{cases} \quad (33)$$

Then the following MILP model (34) can be obtained:

$$\begin{cases} \min & \sum_{t \in \mathbb{T}} \left[\eta \sum_{(i,j) \in A_t} \zeta_{ij} x_{ij}^t + \sum_{i \in S_t^d} c_i \sum_{j \in S_t} x_{ij}^t + \sum_{w \in \mathbb{W}_t} p^{tw} \sum_{i \in S_t} \left(\rho (q_i^{tw})^+ + \delta (h_i^{tw})^+ \right) \right] \\ \text{s.t.} & \text{constraints (2)–(11)} \\ & \text{where, for all } w, \\ & \text{constraints (13)–(14), (16)–(17), (20)–(22), (24)–(26), (28), (30)–(33)} \end{cases} \quad (34)$$

Compared to the original mixed-integer nonlinear programming model, when the scale of the problem is within an acceptable range, the converted MILP model can be solved by using some existing efficient MILP solvers (e.g., CPLEX, Gurobi).

3.2.4. Complexity analysis

The complexity of the MILP model (34) among the entire time horizon is analyzed to facilitate the design of an appropriate solving method, which primarily depends on the number of periods T , the number of stations N , and the number of scenarios W . The decision variables can be categorized into three main groups: route decision variables ($\mathbf{x}$, $\mathbf{z}$) responsible for controlling the travel route of repositioning vehicles, capacity decision variables ($\mathbf{q}$, $\mathbf{y}$, $\mathbf{a}$, $\mathbf{h}$) that control the quantity of loaded/unloaded bikes, available capacity of vehicles, and unmet loading/unloading demands, and time-dependent variables ($\boldsymbol{\tau}$). The complexity of the considered DBRP-SD is evident, even with small-scale instances. As an illustrative example, let us consider 20 stations, 12 periods, and 3 scenarios. The numbers of decision variables and constraints amount to 45,732 and 55,476, respectively. The presence of a substantial number of decision variables and constraints in a multi-period optimization model significantly increases computational time, posing a considerable challenge for efficient solutions, particularly in practical settings where timeliness in addressing the problem is crucial. In the next section, a solving framework that integrates a rolling horizon framework and a heuristic algorithm will be designed to solve the proposed model.

4. Solution approach

To meet the time-sensitive requirements of managing practical-scale dynamic bike repositioning, the general principle of a method is to apply a rolling horizon framework that integrates a heuristic algorithm for solving sub-problems. As a result, Section 4.1 introduces the proposed RHA, including model decomposition on the multi-period model based on the time dimension and two IRHAs. Section 4.2 elaborates on an enhanced heuristic algorithm for solving the decomposed subproblems by combining a parallel genetic algorithm and variable neighborhood search.

4.1. Rolling horizon approach

The studied DBRP-SD is essentially an extension of the PDVRP, which is widely recognized as a NP-hard problem. Although the linear

programming model can be solved by existing MILP solvers, it may encounter challenges such as lengthy computation times and potential memory overflow. When incorporating dynamic repositioning with multiple period decisions, these challenges become more pronounced, particularly for large-scale problems in a large time span. To deal with the dynamic multi-period bike repositioning problem, the rolling horizon approach (RHA) is designed to improve the efficiency of the solution process [35–37].

RHA is a greedy strategy that decomposes multi-period optimization problem along the temporal dimension, that is, to divide a time horizon with T periods into T sub-problems. The underlying optimization problem can be repeatedly modeled across periods, which can be solved sequentially. Unlike conventional global optimization that considers the entire time horizon $[0, T]$, RHA focuses on decision-making from period t to $t + 1$ when the information is available at period t . The variables from the previous period's optimization model serve as initial information for subsequent iterations. Although the RHA may not yield a globally optimal solution, the repeated optimization at each period achieves a satisfactory solution within a shorter computational time, and correct the deviations in prediction across the time horizon with complex situations. Generally, the conventional RHA may exhibit poor performance when considering a long horizon due to its inherent “greedy” and “short-sighted” characteristics. In this section, we first decompose the multi-period optimization problem and then explore the potential limitation of the conventional RHA. Based on this, two IRHAs are introduced to not only enhance the accuracy of RHA solutions but also cater to the solving requirements of diverse scenario-based problems.

4.1.1. Model decomposition

The multi-period optimization model (34) can be decomposed into individual single-period optimization models, each of which can be solved using MILP solvers or heuristic algorithms. The information related to the sets of departure and arrival stations needs to be updated according to vehicles' routes from periods $t - 1$ and t respectively. To reduce the computational complexity of the model, a new set S_t^e is introduced to indicate stations that can serve as departure stations in period t , instead of S_t^d . For the multi-period model, $S_t^d = S_t^e \cup \{0\}$ holds for all $t \in \mathbb{T}$, where $t > 1$. Within the RHA framework, the departure stations can be updated according to the virtual depots at the end of the previous period. Specifically, the departure and arrival stations can be updated as follows. (1) In the first period, vehicles can only depart from the depot and can arrive at virtual parking stations or the depot. Hence, we have $S_1^e = \{0\}$ and $S_1^a = S_1^i \cup \{N + 1\}$. (2) In intermediate periods, S_t^e and S_t^a are updated according to the vehicles' positions at the end of the previous period $t - 1$ and the current period t , respectively. (3) In the last period, all vehicles head to the depot, thus $S_T^a = \{N + 1\}$.

In addition to this, when solving the sub-problem within each period $t, t > 1$, the decisions regarding the travel routes x_{ij}^{t-1} , the available transport capacity $y_i^{(t-1)w}$ of vehicles, and the unsatisfied demand $h_i^{(t-1)w}$ at each station from the previous period are known and carried forward to the subsequent period. As a result, the inter-period constraints of the multi-period model can be modified.

For the first stage, constraint (8) can be reformulated as Eq. (35), which imposes vehicles dwelling at the virtual depot i from the corresponding station j in the previous period $t - 1$ to visit the station j from the virtual depot i in the current period t . Given that the information related to the previous period x_{ij}^{t-1} is known, this constraint is equivalent to fixing a portion of x_{ij}^t as a constant.

$$x_{ij}^t = x_{ij}^{t-1}, \quad \forall i \in S_t^i, j \in S_t. \quad (35)$$

For the second stage, the inter-period constraints that contain formulas (20)–(23) are reformulated as Eqs. (36)–(38). Eq. (36) limits the same available capacity of vehicles among two adjacent periods at the dwell station. Eq. (37) iteratively updates the unmet demand at each

station. Eq. (38) enforces the feasible domain of the decision variable q_i^{tw} by combining the effects of constraints (18) and (23).

$$y_i^{tw} + q_i^{tw} = y_i^{(t-1)w}, \quad \forall i \in S_t^e, \quad (36)$$

$$h_i^{tw} = Q_i^{tw} + h_i^{(t-1)w} - q_i^{tw}, \quad \forall i \in S_t, \quad (37)$$

$$\min\{0, Q_i^{tw} + h_i^{(t-1)w}\} z_i^t \leq q_i^{tw} \leq \max\{0, Q_i^{tw} + h_i^{(t-1)w}\} z_i^t, \quad \forall i \in S_t. \quad (38)$$

Then, a decomposed model of single-period by removing inter-period constraints from the multi-period model (34) can be written as follows. For each period $t, t \in \mathbb{T}$,

$$\begin{cases} \min & \eta \sum_{(i,j) \in A_t} c_{ij} x_{ij}^t + \sum_{i \in S_t^d} c_i \sum_{j \in S_t} x_{ij}^t + \sum_{w \in \mathbb{W}_t} p^{tw} \sum_{i \in S_t} \left(\rho(q_i^{tw})^+ + \delta(h_i^{tw})^+ \right) \\ \text{s.t.} & \text{constraints (2)–(7), (9)–(11), (35)} \\ & \text{where, for all } w, \\ & \text{constraints (13)–(14), (16)–(17), (24)–(26), (28), (31)–(32), (36)–(38)} \end{cases} \quad (39)$$

The decomposed model offers a significant reduction in complexity compared to the multi-period model. Taking the same example as Section 3.2.4 with 20 stations, 12 periods, and 3 scenarios, the decomposed model reduces the number of decision variables and constraints from 45,732 and 55,476 to 5186 and 4823 respectively. This substantial reduction greatly simplifies the model-solving process.

4.1.2. Improved rolling horizon approach

As mentioned above, the conventional RHA may occasionally yield a sub-optimal solution due to its short-sighted nature. For example, suppose a vehicle has completed a repositioning task and remains at the last station i of the current period t , with no available capacity for loading bikes onto the vehicle. In the subsequent period $t + 1$, if a repositioning task requires loading bikes onto the vehicle that has dwelled at station i , RHA, by solely considering the repositioning demand of the current period, may result in the vehicle remaining idle rather than serving other stations or returning to the depot. Consequently, a penalty cost is incurred for the unmet demand, leading to a sub-optimal repositioning scheme.

To overcome the short-sighted of the conventional RHA, one approach is to extend the single-period RHA into a multi-period RHA, referred to as IRHA, to incorporate more demand information from future r ($1 \leq r \leq T, r \in \mathbb{Z}^+$) periods. Fig. 2 illustrates the conventional RHA and two IRHAs considered in this paper. The hollow blocks represent periods that have not been solved in the current step. The wavy-lined blocks indicate periods for which the solution is to be determined and fixed in the current step. The solid gray blocks denote periods for which solutions have already been determined and fixed. The dashed rectangle represents the optimization model currently being solved for the current block.

As depicted in Fig. 2(b), the IRHA-I differs from the single-period RHA shown in Fig. 2(a) by considering not only the information of the current period but also looking ahead to $r - 1$ periods. For instance, when $r = 3$, the information within period t to $t + 2$ is assumed known, and the optimization model that involves three periods can be solved to reach the 3-period optimal solution. The state of the current three blocks is then fixed. The situation of the current model, fixed blocks, and current blocks are updated accordingly. Subsequently, the current model moves to the next r unsolved block. The iteration will continue until all blocks are fixed. By incorporating information from $r - 1$ future periods into RHA, IRHA-I assists in decision-making and avoids excessively greedy situations, thereby enhancing the conventional RHA. The procedure of IRHA-I is clearly summarized in Algorithm 1. We first define the decomposed subproblem as $P_{l:v} = P_{l:v}^{(S_t^e, h^{t-1}, y^{t-1})}$ from period l to v , where $l - v = r$. The current subproblem considers known inputs

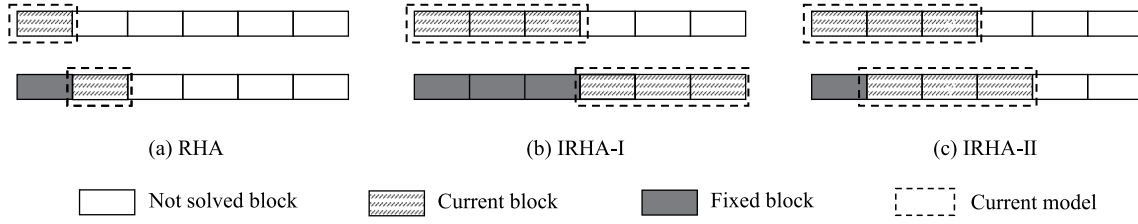


Fig. 2. Rolling horizon approaches, including (a) conventional RHA and (b)–(c) two IRHAs.

from the previous period, including the departure station set S_t^e , the number of unmet demand $h^{l-1} := \{h_i^{(l-1)w} \mid \forall i \in S_{l-1}, w \in \mathbb{W}_{l-1}\}$, and the available capacity of vehicles $y^{l-1} := \{y_i^{(l-1)w} \mid \forall i \in S_{l-1}, w \in \mathbb{W}_{l-1}\}$. Due to the length of periods within multi-period problems may not always be divisible by the horizon r , we set $\beta := \text{Mod}(T, r)$, where $\text{Mod}()$ denotes the modulus operation and calculates the remainder when T is divided by r . Initially, Algorithm 1 solves and fixes the first period problem $P_{1:\beta}$ to optimality. According to this, the algorithm enters a loop to solve each period optimization model sequentially. After solving $P_{l:v}$ from period l to v , the optimal solution set $D = (x_{l:v}, z_{l:v}, \tau_{l:v}, y_{l:v}, q_{l:v}, h_{l:v}, a_{l:v})$ can be obtained.

Algorithm 1 The algorithmic procedure of IRHA-I

Input: Set of periods $\mathbb{T}$, set of bike stations in period t S_t , set of virtual depots in period t S'_t , multi-period problem $P_{l:v}^{(S_t^e, h^{l-1}, y^{l-1})}$, horizon r , initial unmet demand h^0 , initial available capacity y^0

Output: Solution set D of $P_{1:T}^{(S_1^e, h^0, y^0)}$

- 1: $D \leftarrow \{\}$
- 2: $S_1^e \leftarrow \{0\}$
- 3: $\beta \leftarrow \text{Mod}(T, r)$
- 4: $(x_{1:\beta}, z_{1:\beta}, \tau_{1:\beta}, y_{1:\beta}, q_{1:\beta}, h_{1:\beta}, a_{1:\beta}) \leftarrow \argmin P_{1:\beta}^{(S_1^e, h^0, y^0)}$
- 5: $D.add((x_{1:\beta}, z_{1:\beta}, \tau_{1:\beta}, y_{1:\beta}, q_{1:\beta}, h_{1:\beta}, a_{1:\beta}))$
- 6: **for** $k \in \{0, \dots, \lfloor \frac{T-\beta}{r} \rfloor - 1\}$ **do**
- 7: $l \leftarrow rk + \beta + 1$
- 8: $v \leftarrow rk + \beta + r$
- 9: $S_l^e \leftarrow \{i \mid \text{if } x_{ij}^{l-1} == 1, \forall i \in S_{l-1}, j \in S'_{l-1}\}$
- 10: $(x_{l:v}, z_{l:v}, \tau_{l:v}, y_{l:v}, q_{l:v}, h_{l:v}, a_{l:v}) \leftarrow \argmin P_{l:v}^{(S_l^e, h^{l-1}, y^{l-1})}$
- 11: $D.add((x_{l:v}, z_{l:v}, \tau_{l:v}, y_{l:v}, q_{l:v}, h_{l:v}, a_{l:v}))$
- 12: **return** D

Drawing inspiration from Glomb et al. [38], we introduce the second IRHA, referred to as IRHA-II as depicted in Fig. 2(c). In contrast to IRHA-I, IRHA-II does not fix all the current blocks in the current model. Instead, it fixes only a single period after obtaining the r -period optimal solution for the current model. Subsequently, IRHA-II selects r unsolved blocks and proceeds to the next round of solving. The procedure of IRHA-II is summarized in Algorithm 2. Unlike IRHA-I, which “looks multiple steps ahead and fixes multiple steps”, IRHA-II follows a more conservative solving strategy characterized by “looks multiple steps ahead and fixes one step”. Due to its cautious nature, IRHA-II generally takes longer to solve but occasionally yields better solutions than IRHA-I.

4.2. Heuristic algorithm

According to the complexity analysis in Section 3.2.4, it is evident that the complexity of DBRP-SD stems from two key factors: the number of periods T and the number of stations N . In Section 4.1, we employ IRHA to decompose the temporal dimension of the multi-period model, thereby mitigating the complexity associated with the increase in periods. However, the sub-problem within each period is still a NP-hard problem. To address the computational complexity caused by the escalating number of bike stations and to ensure practical feasibility in

Algorithm 2 The algorithmic procedure of IRHA-II

Input: Set of periods $\mathbb{T}$, set of bike stations in period t S_t , set of virtual depots in period t S'_t , multi-period problem $P_{l:v}^{(S_t^e, h^{l-1}, y^{l-1})}$, horizon r , initial unmet demand h^0 , initial available capacity y^0

Output: Solution set D of $P_{1:T}^{(S_1^e, h^0, y^0)}$

- 1: $D \leftarrow \{\}$
- 2: $S_1^e \leftarrow \{0\}$
- 3: **for** $k \in \{1, \dots, T - r + 1\}$ **do**
- 4: $l \leftarrow k$
- 5: $v \leftarrow k + r - 1$
- 6: $(x_{l:v}, z_{l:v}, \tau_{l:v}, y_{l:v}, q_{l:v}, h_{l:v}, a_{l:v}) \leftarrow \argmin P_{l:v}^{(S_l^e, h^{l-1}, y^{l-1})}$
- 7: $S_l^e \leftarrow \{i \mid \text{if } x_{ij}^{l-1} == 1, \forall i \in S_{l-1}, j \in S'_{l-1}\}$
- 8: **if** $k = T - r + 1$ **then**
- 9: $D.add((x_{l:v}, z_{l:v}, \tau_{l:v}, y_{l:v}, q_{l:v}, h_{l:v}, a_{l:v}))$
- 10: **else**
- 11: $D.add((x_l, z_l, \tau_l, y_l, q_l, h_l, a_l))$
- 12: **return** D

solving dynamic repositioning problems for real-world applications, a hybrid heuristic algorithm that combines both genetic algorithm (GA) and variable neighborhood search (VNS) is proposed to tackle the sub-problems.

GA stands out as a group search algorithm that utilizes crossover and mutation operators across the entire population, allowing for the effective exploration of the solution space and exhibiting global search capabilities [39,40]. The solutions are updated iteratively through a selection operation that filters individuals with higher fitness and eliminates those with lower fitness. However, when confronted with intricate optimization problems, GA-based group search algorithms often struggle with converging towards local optima, potentially limiting their ability to achieve the global optimum. VNS, on the other hand, is a meta-heuristic algorithm known for its ability to efficiently explore diverse solution spaces and overcome local optima by employing multiple neighborhood structures and operators during the search process, which has been successfully applied to the PDVRP [41,42]. Each neighborhood operator consists of both “destroying” and “repairing” operations. The “destroying” operation involves modifying the structure of an individual solution to obtain a new solution close to the current one, whereas the “repairing” operation ensures the feasibility of the solution. Therefore, we integrate VNS to enhance the search capability of GA.

To better adapt the improved heuristic algorithm to the characteristics of DBRP-SD, we design specific crossover and mutation operators for GA, as well as neighborhood operators for VNS, all of which can be found in Appendix. Moreover, to leverage the group search capability of GA, increase the solution performance, and maintain a reasonable CPU time when dealing with large-scale populations, we extend the conventional GA into a parallel genetic algorithm (PGA). By harnessing the multi-population search capability of PGA, the accuracy and robustness of problem-solving can be improved.

Ultimately, by embedding a heuristic algorithm within IRHA, a solution framework is developed for the studied DBRP-SD in BSS that can be applied to practice-scale cases. In comparison to state-of-the-art heuristics, our solution framework, named PGA-VNS, integrates the strengths of both group and single-point search algorithms within the DBRP-SD. Specialized operators are designed to enhance the search capability. Parallel computing is then applied to bolster diversity and efficiency in the search process. Aiming to solve each r -period model of IRHA, PGA-VNS involves four main steps: (1) Initialization: The initial step involves population initialization, recording period parameters, and maintaining a set of virtual depots. (2) Neighborhood generation: By conducting PGA-VNS iterations for the initial population, neighborhoods are generated, providing diverse exploration within the solution space. (3) Optimal individual selection: The optimal individual is selected from neighborhoods as the current solution in the next iteration, ensuring the identification of the closer-to-global-optimal solution. (4) Solution update: The current optimal solution is recorded, and the period parameters and departure station set are updated iteratively until the termination condition is satisfied.

4.2.1. Initialized solution

In both GA and VNS, a well-constructed feasible route significantly improves the search efficiency, which in turn facilitates the identification of improved routes. As analyzed earlier, the solution primarily comprises travel routes, arrival times at each station, and transport capacities of vehicles. For our two-stage model, a natural approach is to initialize the first-stage decisions concerning vehicle routes and arrival times, and subsequently derive the remaining decisions by solving the second-stage programming model. Building upon this analysis, Algorithm 3 presents the procedure for generating initial feasible routes.

Algorithm 3 Initialization of feasible routes

Input: Station number N , scenario number Ω , station flow data Q_i^{tw} , time window $[L_i, R_i]$, horizon $r, r \leq T$, virtual station set S_i^e

Output: Set of feasible routes $Routes$

```

1:  $Routes \leftarrow \{\}$ 
2:  $virtual\_station\_nums \leftarrow$  The length of  $S_i^e$ 
3: for  $k \in \{1, \dots, r\}$  do
4:    $candidate\_set \leftarrow$  Randomly distribute stations
5:    $vehicle\_number \leftarrow$  Estimated  $candidate\_set$  time
6:   if  $virtual\_station\_nums == 0$  then
7:      $temporary\_path \leftarrow$  Generation a station set randomly slice the
        $candidate\_set$  based on the  $vehicle\_number$ 
8:      $route \leftarrow$  Add the depot as the starting station for  $temporary\_path$ 
9:   else  $virtual\_station\_nums != 0$ 
10:     $temporary\_path \leftarrow$  Generation a station set randomly slice the
       $candidate\_set$  based on the  $vehicle\_number$  and  $S_i^e$ 
11:     $route \leftarrow$  Add the  $virtual\_station$  as the starting station for
       $temporary\_path$ 
12:    generate  $\mu \sim Uniform[0, 1]$ 
13:    if  $\mu \leq returnrate$  then
14:       $route \leftarrow$  Return depot
15:     $route\_time \leftarrow$  Check out the time of  $route$ 
16:    if  $route\_time \in [L_i, R_i]$  then
17:      Continue
18:    else
19:       $route \leftarrow$  Splitting  $route$ 
20:     $Routes.add(route)$ 
21:    update  $S_i^e$ 

```

The candidate feasible routes are created by randomly shuffling the candidate stations. The technical details are as follows. The number of vehicles dispatched from the depot is firstly estimated based on travel

times and repositioning demand among candidate stations, with random perturbations applied for variability. Then the candidate stations are randomly grouped according to the estimated number of vehicles to generate candidate travel routes. The departure and arrival stations are determined by matching stations corresponding to virtual depots in the departure station set S_i^e . For each candidate travel route, the arrival time at each station is calculated. In cases where time window constraints are violated, stations on the long route can be divided into two groups, and then generate two feasible routes, whose departure stations are selected from the departure station set. Once the time window constraints are satisfied, vehicles have a certain probability of returning directly to the depot instead of proceeding to the virtual depot to increase the diversity of the generated initial solutions. Each single route is extended to multiple routes for multiple periods by updating the set of departure stations, ensuring alignment between routes in adjacent periods.

After generating the feasible routes and the corresponding arrival times at stations for the first-stage problem, two approaches that calculate the load/unload quantities on vehicles corresponding to each route are introduced. The first approach involves fixing the decision variables of the first stage and utilizing a MILP solver to solve the second-stage model, which is formulated as follows:

$$\begin{cases} \min & \sum_{i \in \{k, k+r-1\}} \sum_{w \in W_i} p_i^{tw} \sum_{i \in S_i} \left(\rho(q_i^{tw})^+ + \delta(h_i^{tw})^+ \right) \\ \text{s.t.} & \text{where, for all } w, \\ & \text{constraints (13)–(14), (16)–(17), (24)–(26), (28), (30)–(33)} \end{cases} \quad (40)$$

In this case, variables x_{ij}^t , z_i^t , and τ_i^t are known from Algorithm 3. However, when the number of periods and stations increases, this approach still imposes a significant computational burden. As a result, we employ an alternative heuristic strategy to derive the capacity-related decision variables y_i^{tw} , q_i^{tw} , h_i^{tw} , and a_i^t . Firstly, we randomly select an initial available capacity for a vehicle within the range of $[0, E]$. Based on the pre-determined routes and repositioning demand at each station, the available capacities and load/unload quantities are greedily calculated. When the available capacity falls short of meeting the repositioning demand at a particular station, the unsatisfied demand is categorized as unmet demand. This process works for obtaining a feasible and greedy solution for the second-stage model, alleviating the computational complexity with a large number of periods and stations.

4.2.2. Parallel computing strategy

To further improve the solution performance and save computational overhead, multi-core CPUs are utilized in our solving framework to parallelize serial computations. With Python's multi-core computing toolkit Multi-Processing, we establish a parallelized GA-VNS framework as shown in Fig. 3. Two main aspects are improved with parallel computing.

For GA, the large initial population can be divided into multiple smaller sub-populations, which are performed by evolutionary operations independently in parallel. By assigning different mutation probabilities to these sub-populations, each of which has different search capabilities and objectives, so that sub-populations with large variance can explore the global optimal solution effectively, while sub-populations with small variance can focus on searching for the current local optimal solution. By making a trade-off between “exploration” and “exploitation” of GA, PGA can effectively utilize the available computational resources and enhance the population search capability. After several iterations of sub-populations, a population information-sharing mechanism is built to transfer the optimal individuals from each sub-population to the entire population, thus incorporating the advantageous genes of the sub-populations.

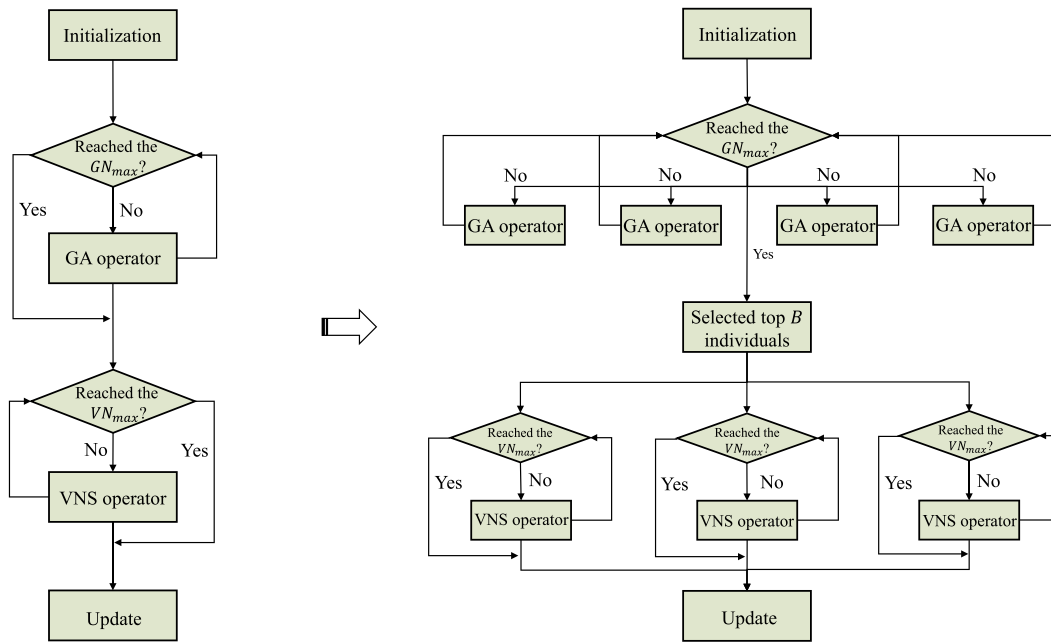


Fig. 3. The structure of the parallel GA and VNS.

For VNS, to leverage its single-point search capability, the top B diverse solutions obtained from PGA are extracted as the input of the VNS algorithm in parallel respectively, so as to better utilize the different individual information searched by the GA population to achieve better performance.

4.2.3. Neighborhood structures

Our neighborhood generation process includes two components, evolutionary operations from GA and variable neighborhood from VNS. For GA, after generating initial feasible solutions by Algorithm 3 to obtain each sub-population, we proceed to perform evolutionary operations, including selection, crossover, and mutation, on each sub-population in parallel. The designed crossover and mutation operators are elaborated in Appendix A.1. Each sub-population is assigned its own operators and variance probabilities. Once the maximum number of iterations is reached, PGA stops, and the sub-populations from different CPUs are merged to form the final optimal population set.

For VNS, as illustrated in Fig. 3, top B different solutions are accepted from PGA as the inputs. There are two basic ingredients to improve searching ability involving shaking and variable neighborhood descent (VND). In the shaking phase, one of the ten designed destroy-repair operators (refer to Appendix A.2) is randomly chosen to perturb the initial feasible solution, generating the current solution, which serves as the input for the subsequent VND process. In the VND phase, the current solution is iteratively improved by sequentially using neighborhood operators within its neighborhood space. If a neighborhood solution is found to be superior to the current solution, it is accepted as the new current solution, and the initial neighborhood operator is reset. Otherwise, the next operator is employed. This process continues until all the operators have been executed. Finally, among the B solutions obtained from each CPU, the one with the smallest objective value is considered the optimal solution from the PGA-VNS output in the current period.

4.2.4. Framework of PGA-VNS

After obtaining the optimal solution for the current period, the information related to the period involving sets of departure and arrival stations, unmet demands, and available capacities of vehicles is updated and rolled to the next r -period. The PGA-VNS algorithm is then applied iteratively to solve the model until the last period model is

solved. The detailed procedure of the PGA-VNS framework is outlined in Algorithm 4.

5. Computational tests

In this section, we conduct computational tests to evaluate the performance of our proposed model and solution approach based on the real-world data obtained from BSS in Beijing. All experiments are implemented on a laptop equipped with AMD Ryzen 7 5800H processor with Radeon Graphics 3.20 GHz, 16 GB RAM, and running Microsoft Win11 Pro OS. The MILP model is solved by Gurobi 9.5.0, with a termination condition set at a maximum CPU time of 3600 s.

5.1. Data preparation

For this study, we gathered approximately 7 million travel records from BSS in Beijing, China from March 24, 2018, to March 30, 2018, and from May 18, 2018, to May 31, 2018, which contain Order ID, Start time, End time, Start longitude, Start latitude, End longitude, and End latitude. Table 4 exemplifies a few collected records.

A pre-processing step is employed for data processing and extracting relevant information for dynamic repositioning in the BSS. Some outliers are eliminated from the original data, such as instances where travel times or latitude and longitude values are equal to 0. The focused area of interest is located within the Xicheng District of Beijing. To account for the temporal dimension, we divide the daytime horizon into hourly intervals, starting from 9:00 and ending at 21:00, resulting in a total of $T = 12$ periods. In practical BSS operations, users are typically required to park bikes within electronic fences, which are not available in our data. Therefore, we partition the researched area into 280 grids, each with a size of 500 m. In this grid-based approach, the coordinates of the grid's center are regarded as the position of the station. The start and end latitude and longitude coordinates from the travel record data are counted to calculate the gap between the outflow and the inflow of bikes corresponding to each grid during each period. This information serves as an indicator for calculating the repositioning demand at each station.

To recognize multiple scenarios from the history travel record data, we use K-means clustering to divide the daily demand into several scenarios by considering exogenous variables such as total inflow and

Table 4

Example of Beijing BSS travel record data.

Order ID	Start time	End time	Start longitude	Start latitude	End longitude	End latitude
MBK01068313561526570713449	20180517232517	20180517232538	116.396440	39.945755	116.391730	39.944004
MBK01065057361526569485630	20180517230447	20180517232547	116.299240	39.959827	116.284900	39.935017
MBK01065991991526566937527	20180517222219	201805172222912	116.454870	39.874653	116.444980	39.871304
MBK01002059431526569143151	20180517225919	20180517230025	116.547386	39.758980	116.547620	39.758644
1760894901600106245	20180517202401	20180518000001	116.367970	40.020393	116.365770	40.031345
189462591562267059	20180517235243	20180518000001	116.426070	39.864494	116.429470	39.857300

Algorithm 4 The framework of PGA-VNS

Input: Number of periods T , horizon $r, r \leq T$, station number N , scenario number $\mathbb{W}$, bike station demand Q_i^{tw} , time window $[L_i, R_i]$, GA maximum number of iterations GN_{max} , GA population number K , GA crossover probability P_c , GA mutation probability P_m , VNS maximum number of iterations VN_{max} , destroy-repair operator list Ns , number of parallel populations S , optimal number of individuals B

Output: Optimal solution and optimal objective function value for each period

```

1:  $CPUtime \leftarrow 0$ 
2:  $k \leftarrow 1$ 
3:  $initial\_population \leftarrow []$ 
4: for  $k \leq K$  do
5:    $solution \leftarrow \text{Initialization}(N, \Omega, Q_i^{tw}, L_i, R_i)$ 
6:   put  $solution$  in  $initial\_population$ 
7:  $current\_subpopulations \leftarrow []$ 
8: for  $s \leq S$  do
9:    $subpopulation \leftarrow \text{Split population}(initial\_population)$ 
10:  put  $subpopulation$  in  $current\_subpopulations$ 
11:  $i \leftarrow 1$ 
12: while  $i \leq GN_{max}$  and  $CPUtime \leq MaxTime$  do
13:    $new\_subpopulations \leftarrow \text{PGA}(T, N, S, \Omega, Q_i^{tw}, L_i, R_i, K, P_c, P_m, current\_subpopulations)$ 
14:    $current\_population \leftarrow \text{Information sharing}(new\_subpopulations)$ 
15:    $new\_best\_solution \leftarrow$  The best solution in  $new\_population$ 
16:   if  $new\_best\_solution \leq best\_solution$  then
17:      $best\_solution \leftarrow new\_best\_solution$ 
18:      $best\_population \leftarrow current\_population$ 
19:    $i \leftarrow i + 1$ 
20:   update  $CPUtime$ 
21:  $top\_B\_solutions \leftarrow$  Top  $B$  solution in  $best\_population$ 
22:  $current\_solutions \leftarrow top\_B\_solutions$ 
23:  $best\_solution \leftarrow$  The best solution in  $current\_solutions$ 
24:  $j \leftarrow 1$ 
25: while  $j \leq VN_{max}$  and  $CPUtime \leq MaxTime$  do
26:    $new\_solutions \leftarrow \text{PVNS}(T, N, \Omega, Q_i^{tw}, L_i, R_i, Ns, current\_solutions)$ 
27:    $new\_best\_solution \leftarrow$  The best solution in  $new\_solutions$ 
28:    $current\_solutions \leftarrow new\_solutions$ 
29:   if  $new\_best\_solution \leq best\_solution$  then
30:      $best\_solution \leftarrow new\_best\_solution$ 
31:    $j \leftarrow j + 1$ 
32:   update  $CPUtime$ 
33: update period

```

outflow of bicycles in 12 periods and temperature. Specifically, the 21 days are clustered into three categories: weekends (scenario 1), weekdays with higher temperature (scenario 2), and weekdays with lower temperature (scenario 3). The distribution of days within each scenario results in probabilities of 0.2857, 0.4286, and 0.2857, respectively. Fig. 4 illustrates the active area of the three scenarios during the off-peak period of 11:00–12:00, the evening peak period of 17:00–18:00,

and the post-peak period of 19:00–20:00. In scenario 1, the distribution of user demand is relatively consistent in both time and space, whereas in scenarios 2 and 3, there are noticeable tidal characteristics with distinct evening peaks, primarily concentrated around residential areas and commercial districts. The most significant disparity is observed between scenarios 2 and 3 during peak periods, taking into account the likelihood of individuals altering their transportation preferences due to temperature variations. Consequently, the demand for shared bikes is considerably higher in suitable temperatures (23 °C on average) compared to relatively lower temperatures (15 °C on average).

Other parameter settings in the DBRP-SD model are elaborated as follows. Suppose the maximum available number of vehicles in the depot is $m_0 = 30$ and in virtual depots are $m_i = 1$, $i \in S'_i$, each with a maximum capacity $E = 100$. The dispatch cost c_i from the depot is 50 and from virtual stations is 10. The velocity of vehicles is assumed to be 30 km/h, while the unit traveling time cost η is set to 0.5/min. We set the unit bike repositioning cost $\rho = 0.5$ and the unit bike repositioning service time $\mu = 0.1$. The unmet demand penalty cost $\delta = 5$ is particularly set to guarantee the service quality. The distance between two stations can be obtained through Google Maps according to the positions of the two stations.

5.2. Parameter calibration on key coefficients

In this section, we calibrate the value of δ that represents the penalty coefficient of capacity supply shortage for unmet repositioning demand, which would decrease the system performance if it is set unreasonable. A small δ incurs a large quantity of unmet demand, while a large δ restricts the system's flexibility by requiring all repositioning demands to be satisfied. To assess the impact of different values of δ , three sets of small-scale examples are implemented under conventional RHA (strategy shown in Fig. 2(a)), IRHAs (strategies shown in Fig. 2(b) and (c)) and All periods that to solve the MILP model (39) directly. These examples are performed with five stations from the grid data. The coordinate of the depot is [39.945261, 116.352994] and of the five stations are [39.878034, 116.335765], [39.968033, 116.329765], [39.920034, 116.323765], [39.902034, 116.359765], and [39.938034, 116.323765]. The repositioning demands at different stations under three scenarios and twelve periods are illustrated in Table 5, where periods 1, 2, ..., 12 represent [9:00, 10:00], [10:00, 11:00], ..., [20:00, 21:00], respectively. To ensure the optimality of the MILP model (39) within a short computational time, the first four periods are considered in this experiment setting. Other parameters are kept the same as in Section 5.1. The value of δ varies from 0 to 10 with an increment of 0.4. All solutions are obtained by Gurobi.

In Fig. 5, the red solid line represents the operational cost corresponding to the left vertical axis, while the blue dotted line represents the number of unmet demands (indicated as UD) corresponding to the right vertical axis. As the value of the penalty coefficient rises, the quantity of UD initially drops sharply and then levels off. To obtain the solution with the lowest UD across all three cases, we choose $\delta = 5$ as illustrated by the gray dash-dotted line in Fig. 5. Furthermore, we observe that the short-sightedness of RHA results in an inability to fully satisfy the demand in all periods. This leads to a linear increase in the total cost of RHA with a higher penalty coefficient. On the other hand,

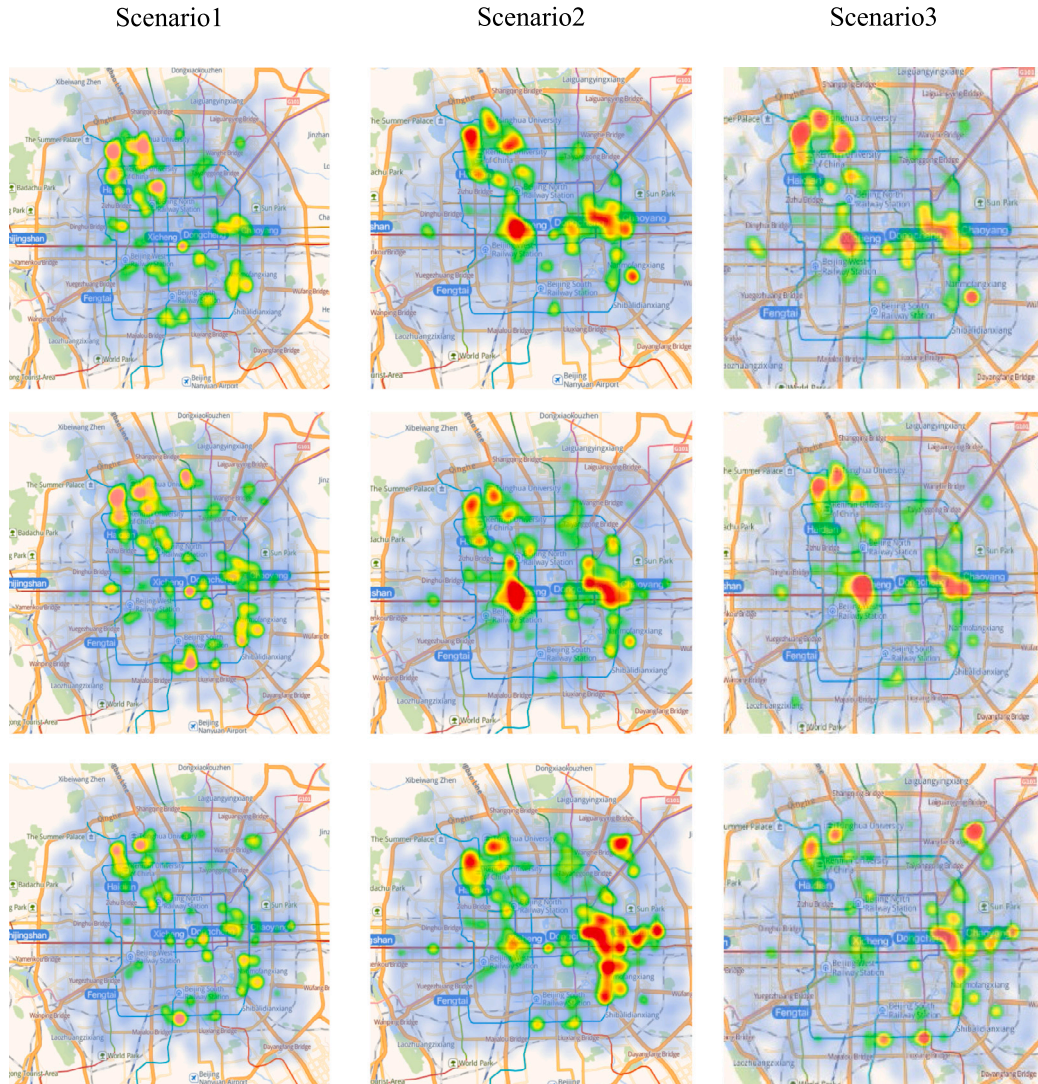


Fig. 4. Demand spatial distribution in different scenarios and periods.

Table 5
Repositioning demands at five stations under three scenarios and twelve periods.

Stations	w	Periods											
		1	2	3	4	5	6	7	8	9	10	11	12
1	1	-16	-1	0	0	0	0	0	0	0	0	0	16
	2	-3	0	-1	1	0	-1	0	0	1	-1	-1	21
	3	0	-1	0	0	0	0	0	0	0	0	0	10
2	1	-8	13	14	3	5	-4	-2	3	4	-5	3	12
	2	-1	13	18	-2	5	3	5	13	94	40	25	27
	3	1	10	12	-3	7	0	4	17	84	26	13	19
3	1	-16	-17	-6	-3	-2	-5	-4	4	-10	-1	5	-18
	2	-11	-19	-9	-10	5	-3	-2	1	6	1	0	-9
	3	-7	-1	-9	-7	-5	-2	7	14	12	9	-1	0
4	1	-6	-10	-1	-1	20	7	19	11	-9	5	5	14
	2	13	-77	0	13	26	9	42	125	-113	-22	-11	1
	3	-7	-67	-6	12	16	8	42	92	-104	-27	-4	13
5	1	-4	-2	2	1	-3	2	-4	-8	14	14	11	15
	2	1	0	6	7	-2	-16	-37	-41	42	23	18	20
	3	7	-1	5	6	4	-19	-29	-35	47	39	20	20

the IRHA framework significantly mitigates this problem encountered with RHA, enabling the satisfaction of demand through bike repositioning. This suggests that companies prioritizing user satisfaction are less likely to utilize the conventional RHA for bike repositioning.

5.3. Analysis of mathematical model

To analyze the performance of the proposed mathematical model, which incorporates multiple scenarios and the VNR strategy, we conduct two sets of comparison experiments in the following sub-sections.

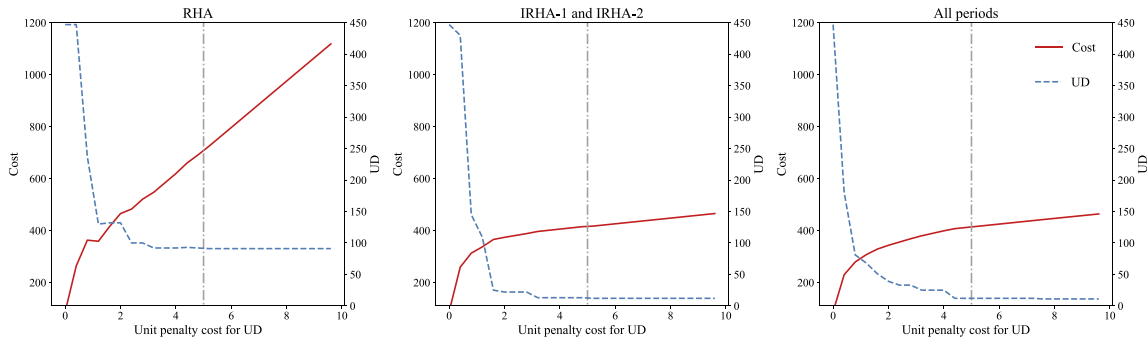


Fig. 5. Comparison of costs and UD with different penalty coefficients.

5.3.1. Comparison between scenario-based stochastic and determined model

To validate the advantages of considering the scenario-based stochastic model, we compare solutions derived from a single-scenario model with those obtained from our scenario-based stochastic model. In this experiment setting, five stations and twelve periods are considered, while other settings remain the same as Section 5.2. Both sets of experiments are implemented under the IRHA-I framework, when the parameter r is set to 1, 2, 3, 4, 6, and 12 respectively. Increasing r represents considering more future information when making decisions for the current period. When $r = 1$, the IRHA framework degenerates to the conventional RHA; when $r = 12$, the entire multi-period model is solved without rolling strategies.

The single-scenario demand is calculated by taking the weighted average of the demand across three scenarios. The number of UD is obtained by applying the repositioning plan derived from the determined single-scenario to all three scenarios. Fig. 6 presents the comparison between the scenario-based stochastic model (pink bars) and the determined model (blue bars) when applying IRHA-I with $r = 3$. Notably, the deterministic model causes enormous UD that is related to the tidal characteristics of bike-sharing demand. During the evening peak hours at 17:00 for example, the UD reaches a significantly higher level compared to other periods. Regardless of the specific scenario, UD persists across all periods, even when implementing the framework. When solving the scenario-based stochastic model, a significant reduction in UD occurs. Scenario 1 (Fig. 6(a)) shows no UD, while scenarios 2 and 3 (Fig. 6(b)–(c)) exhibit lower levels of UD during the evening peak period. This outcome highlights the effectiveness of the scenario-based stochastic model in enhancing the robustness of dynamic repositioning across multiple scenarios and improving overall system stability.

5.3.2. Analysis of VNR strategy

As summarized in Table 1, few studies have considered the VNR strategy, which allows vehicles to dwell at the last visited station in the current period instead of returning to the depot. Therefore, this section aims to highlight the advantages of applying the VNR strategy through the visualization presented in Fig. 7. The accumulated operational cost is illustrated in Fig. 7(d), which is composed of three components: traveling cost (Fig. 7(a)), dispatching expense (Fig. 7(b)), and penalty cost (Fig. 7(c)). The dashed blue line with triangle symbols denotes the scenario where all vehicles return to the depot at the end of each period, while the solid red line with circle symbols denotes the scenario where the VNR strategy is applied. Despite a minor increase in penalty cost, the utilization of the VNR strategy performs better with substantial savings in traveling cost and dispatching expense, which significantly reduces the accumulated operational cost by 29.04%. Consequently, applying the VNR strategy in the dynamic bike repositioning aligns better with practical applications, and effectively reduces the comprehensive cost.

5.4. Analysis of solution approach

According to Sections 4.1 and 4.2, an IRHA framework is introduced to decrease the temporal dimension complexity of the multi-period model, while a hybrid PGA-VNS heuristic is proposed to solve the sub-problem that is still a NP-hard problem. In this section, the performance of different IRHAs and heuristic algorithms are assessed to demonstrate the effectiveness and efficiency of our proposed approach.

5.4.1. Comparison of rolling horizon framework

In addition to the conventional RHA, two IRHAs are proposed to deal with the dynamic characteristic of the studied DBRP-SD. In the rolling horizon framework, the entire time horizon is divided into a set of periods, each of which is solved using either a commercial solver or a heuristic algorithm. To evaluate the effectiveness of the two IRHAs, specifically, IRHA-I and IRHA-II, the objective values and CPU times among r -periods are regarded as the evaluation criteria, which allow us to assess the improvements in terms of optimization performance and computational efficiency. The experimental settings remain the same as in Section 5.2 as well. Due to the characteristic of “multiple decisions” in IRHA, the error of a single decision may accumulate and amplify during rolling. Therefore, an indicator of the “cumulative gap” is introduced that represents the cumulative value of the solving algorithm’s errors across multiple periods within IRHA. When the “cumulative gap” equals 0, it indicates an absence of solving errors. In this regard, the only disparity arises from rolling errors inherent to the approach. When the “cumulative gap” is greater than 0, additional error accumulates due to each subproblem’s solving error throughout the multiple rolling steps. The specific experimental results are presented in Table 6.

When $r = 1$, both IRHA-I and IRHA-II degrade to the conventional RHA; when $r = 12$, both IRHA-I and IRHA-II are equivalent to the multi-period optimal model without rolling strategies. Obviously, IRHA-I and IRHA-II exhibit different precision and efficiency properties with different values of r . Compared to the conventional RHA (i.e., $r = 1$), IRHAs demonstrate improved solution performance with smaller objective values as they incorporate future information in each period when $r < 4$ with lower time complexity per iteration. When $r > 6$, the optimization model with more than 6 periods becomes increasingly complex to solve where the optimal solutions cannot be reached with the pre-set maximum computational time.

Specifically, IRHA-I can be utilized for achieving more stable solution results, striking a better balance between solution performance and efficiency. Compared to the conventional RHA, IRHA-I sacrifices a certain degree of efficiency in exchange for higher solution performance. By choosing an appropriate value of the parameter r , it is possible to achieve superior solution performance compared to RHA within an acceptable solving time. IRHA-II adopts a “looks multiple steps ahead and fixes one step” scheme, which results in longer CPU times as it involves a total of 12 iterations, while $12/r$ iterations for IRHA-I. As the value of r increases, the decomposed multi-period problem becomes

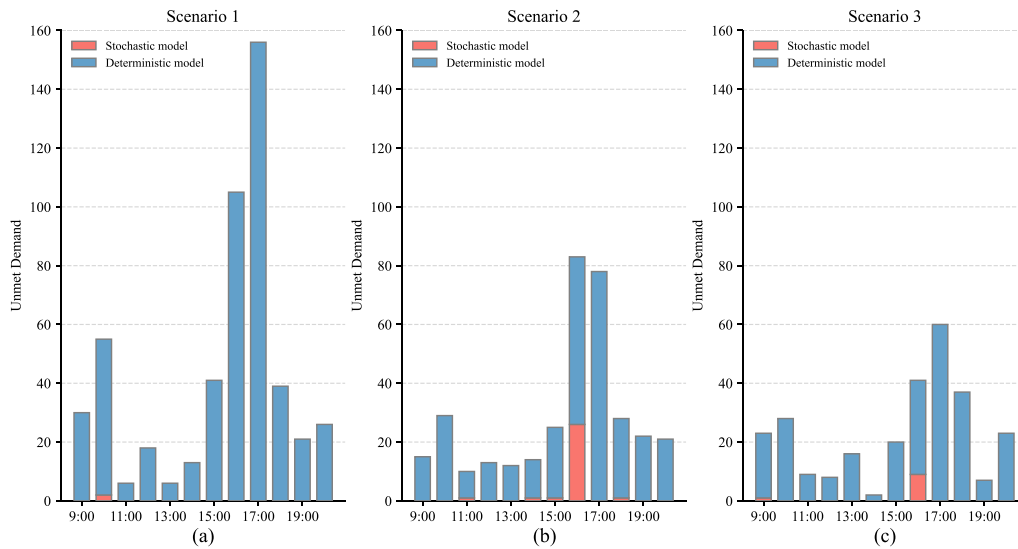


Fig. 6. Comparison between stochastic and determined model within different periods. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

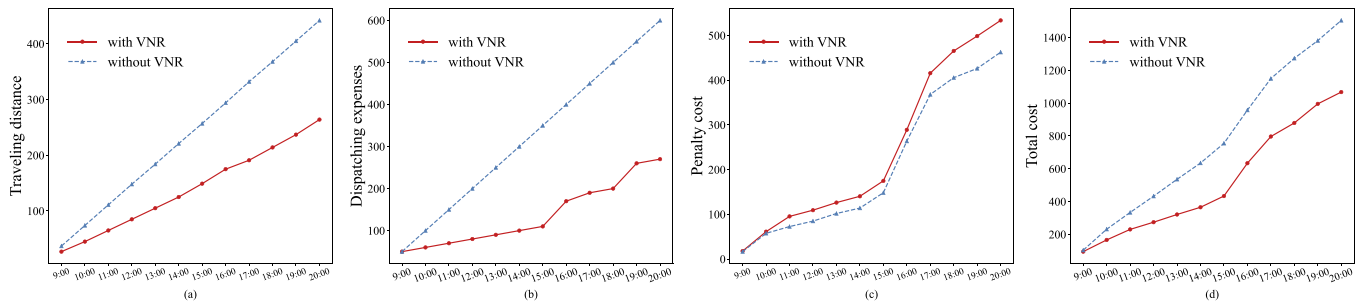


Fig. 7. Performance of considering VNR strategy in terms of the accumulated (a) traveling cost, (b) dispatching expense, (c) penalty cost, and (d) total operational cost. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 6
Comparison of different IRHAs with different r .

r	IRHA-I			IRHA-II		
	Cost	CPU time (s)	Cumulative gap	Cost	CPU time (s)	Cumulative gap
1	1362.79	0.84	0%	1362.79	0.84	0%
2	1102.50	1.57	0%	1061.64	2.60	0%
3	1053.50	10.49	0%	1113.93	10.50	0%
4	1051.50	101.06	0%	1056.07	255.81	0%
6	1099.21	>3600	2.26%	1096.36	>3600	2.02%
12	1086.79	>3600	5.97%	1086.79	>3600	5.97%

more complex, incurring highly inefficient solving times and higher cumulative gaps. In cases where the horizon is smaller (e.g., $r = 2$), the complexity in the decomposed multi-period sub-problems is lower, where IRHA-II often exhibits superior properties. However, for bigger horizons, longer CUP times are generated, while the performance cannot be guaranteed compared to IRHA-I.

As discussed above, the value of r highly influences the performance of the two IRHAs. When determining the value of r , it is important to consider not only the solution performance and the solving efficiency but also the rolling errors and prediction errors of future information. Fig. 8 presents the trends of the rolling error (red dashed line), the cumulative error (blue dashed line), and the total operational cost (green solid line). The prediction error is supposed to increase as the number of periods accumulates. The rolling error is calculated by comparing the objective function with the entire period optimization model. As the value of r increases, the rolling error gradually decreases, while

the prediction error gradually increases. In addition to this, the total cost exhibits a pattern of decreasing costs followed by increasing costs. When r is small, the rolling error dominates the overall error, whereas when r is large, the prediction error becomes more significant. Therefore, both excessively large and small values of r lead to higher errors and costs for the repositioning scheme in practical operations. According to the result shown in Fig. 8, a moderate horizon value (typically 2–3) can be chosen to strike a balance between the prediction error and the rolling error within our IRHA framework.

5.4.2. Comparison of heuristic algorithm

In this section, we compare the performance of the proposed hybrid PGA-VNS algorithm with a high-performance commercial solver Gurobi 9.5.0, and other benchmark heuristics, including GA, GA with Gurobi, VNS, and PGA. Some parameters related to the involved algorithm are set as follows. For the GA-related algorithm, 8 cores are considered in the parallel computing. The population size is set to 50. The crossover

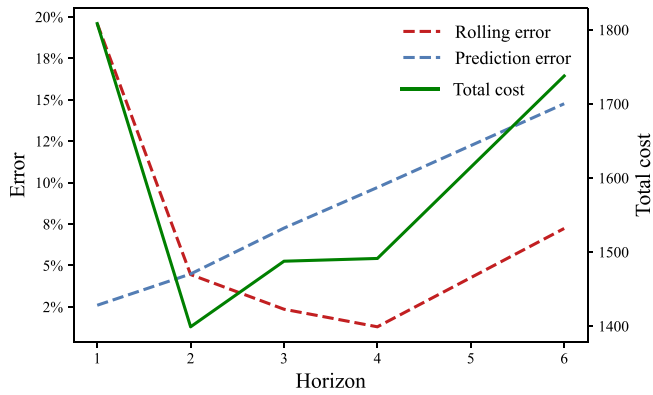


Fig. 8. Results of rolling error and prediction error under different values of r . (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

probability is 0.9, and the mutation probability is 0.2. For the hybrid PGA-VNS algorithm, the number of input solutions from PGA B is 8. Each sub-population is assigned a mutation probability ranging from 0.05 to 0.45 with step increments of 0.05. All algorithms are executed for 50 iterations. Our evaluation focuses on the solution performance and solving efficiency depending on illustrative numerical examples, and the results are summarized in Table 7. Taking the practical scenario into consideration, a maximum solving time of four hours is established. If the algorithm exceeds this limit without finding a solution, it is considered unsuitable for practical implementation. In Table 7, such cases are denoted with “–” for Cost and “>3600” for CPU time.

For small-scale instances, the Gurobi solver quickly reaches the optimal solution, serving as a benchmark for comparison. The proposed PGA-VNS algorithm achieves the optimal solution when considering five stations, and obtains a near-optimal solution with a small gap of 1.92%, while improving CPU times by 34.8% when considering ten stations. However, as the number of stations increases, the Gurobi solver is unable to complete the solution within a reasonable time frame. For moderate- and large-scale instances, PGA-VNS outperforms other heuristic algorithms by achieving the best solution performance with the lowest objective function within a reasonable time frame. Notably, GA and VNS can quickly calculate lower-performance solutions, the gaps between them and PGA-VNS are up to 57.63% and 36.57%, respectively. GA with Gurobi and PGA yield better solutions compared to GA and VNS, but with increased CPU times. In addition, comparing GA and PGA reveals that the parallel framework significantly improves solution performance at the expense of solving efficiency. The comparison between PGA and PGA-VNS demonstrates the effectiveness of VNS in exploring diverse solution spaces.

In summary, the proposed PGA-VNS algorithm consistently provides high-quality solutions compared to the basic GA and VNS methods across all cases. The shaking operation in VNS generates more neighborhood structures, enabling the exploration of diverse solution spaces and yielding better solutions than the PGA algorithm. It is evident that, for all instances, the PGA-VNS algorithm produces high-quality solutions, outperforming the Gurobi solver and other heuristics.

5.5. A real-world case in Beijing

To investigate a real-world case, we examine a specific location in Xicheng District, Beijing, over a span of twelve periods, starting from 9:00 and ending at 21:00. The repositioning vehicles commence their operations from the depot at the start of the day, carrying out repositioning tasks, and subsequently return to the depot at the end of the day. As explained in Section 5.1, the research area is partitioned into 280 grids. However, not all stations are visited by a vehicle

during each period. On the one hand, some grids may be located in parks where shared bikes are not permitted to be parked. On the other hand, there may be a slight disparity between inflow and outflow in certain grids, which is not necessary to be served. Consequently, in this real-world experimental setup, stations with repositioning demands below a certain threshold (5 in this experiment) are excluded from the current period's repositioning activities to save the repositioning resources to the urgent stations. The parameter setting for the model and heuristic algorithm remain consistent with those outlined in Section 5.1 and Section 5.4.2. The scenario-based stochastic model, incorporating the VNR strategy, is solved by the two IRHA frameworks ($r = 2$) combined with the hybrid PGA-VNS, addressing the complexities of the DBRP-SD.

Table 8 provides a comprehensive overview of the optimized repositioning results for the twelve periods, encompassing the number of stations, vehicles, served bikes, and UD. The operational costs are presented as well including the travel time cost, vehicle dispatched expense, delivery cost, penalty cost, and total cost respectively. During the peak periods 8–10, i.e., the evening rush hours between 16:00 to 19:00, the demand for repositioning is significantly higher compared to other periods. More vehicles are dispatched for these repositioning tasks, leading to increased expenses in total operational costs. For example, in the case of IRHA-I, during peak period 9 (17:00–18:00), 24 vehicles are dispatched to visit a total of 122 stations and reposition 2669 bikes. During an off-peak period like period 5 (13:00–14:00), only 3 vehicles are dispatched to reposition 84 bikes.

Service rate, defined as the proportion of served bikes relative to the sum of served bikes and UD, is regarded as a performance metric. The IRHA-I framework exhibits shorter CPU time and greater robustness in handling demand surges, meanwhile, the repositioning service rate achieves an average of 95.00%. In comparison, the IRHA-II exhibits significantly longer CPU times, almost double that of IRHA-I. However, in certain periods with stable demand, IRHA-II outperforms IRHA-I. For instance, during periods 1–6, the number of served bikes when implementing IRHA-II is higher than that of IRHA-I. However, the situation is entirely reversed during periods 7–9. Therefore, IRHA-II struggles to effectively adapt to sudden increases in demand, resulting in significant UD during peak hours and a decrease in the repositioning service rate, averaging 94.95%. Furthermore, the solving time for each period reaches a minute level, underscoring the practical applicability of the proposed approach in addressing dynamic bike repositioning challenges in practical operations.

In general, IRHA-I is a more direct way to integrate future information into RHA. It is simple to implement and yields superior performance. Adjusting the horizon control effectively balances efficiency and accuracy, making it particularly suitable as an improved rolling horizon framework. On the other hand, IRHA-II can be regarded as a more conservative IRHA. Instead of immediately reacting to make decisions upon future information, it takes a “looks multiple steps ahead and fixes one step” strategy. Consequently, with a lower horizon setting, IRHA-II tends to make more conservative decisions which may not perform well in periods of high demand variability. However, when inter-period demand gradually changes, IRHA-II often yields superior outcomes compared to IRHA-I, albeit at the expense of additional computational time. Therefore, businesses should carefully consider both IRHAs based on their specific operational problems and objectives.

5.6. Management implications and insights

Based on the experimental results, we refine and summarize a set of managerial implications and insights for platform operators within the BSS.

Firstly, the efficient operation of BSS is significantly influenced by the spatiotemporal characteristics of repositioning demands. To employ dynamic repositioning to rebalance the BSS, it is imperative to conduct a comprehensive analysis of the historical demand patterns. During peak periods, the platform has to anticipate and carry out preemptive

Table 7
Comparison between exact and other benchmark heuristics.

Number of stations	Gurobi		GA		GA + Gurobi		VNS		PGA		PGA-VNS	
	Cost	CPU (s)	Cost	CPU (s)	Cost	CPU (s)	Cost	CPU (s)	Cost	CPU (s)	Cost	CPU (s)
5	264.43	0.26	333.07	3.27	264.43	41.91	283.43	6.53	293.29	29.42	264.43	31.55
10	517.86	181.45	613.93	8.31	583.79	97.71	557.89	19.64	550.86	93.06	528.00	118.25
15	–	>3600	825.79	10.95	815.21	166.4	791.07	31.46	780.93	139.27	764.45	165.00
20	–	>3600	1425.36	22.76	1198.07	283.98	1343.67	44.81	1361	214.19	1034.64	259.27
30	–	>3600	2705.5	58.36	2030.79	697.95	2343.98	75.71	2124.14	342.79	1716.36	572.71
50	–	>3600	5254.76	168.39	4242.86	1397.7	4314.56	254.78	4570.21	731.66	4013.14	1145.93
100	–	>3600	8616.07	702.40	8354.57	6668.23	8123.23	921.42	7980.07	3323.13	7521.21	4323.13

Table 8
Results of real-world experiment for different periods with two IRHAs.

	Periods											
	1	2	3	4	5	6	7	8	9	10	11	12
IRHA-I												
Number of stations	53	75	58	56	39	61	84	106	122	97	74	77
Number of vehicles	6	7	4	4	3	3	12	20	24	17	5	4
Number of served bikes	396	305	347	378	84	117	922	2746	2669	1450	436	696
UD	9	6	35	32	0	4	70	119	187	73	48	22
Vehicle traveling time cost	203.00	204.00	203.00	158.00	87.00	71.00	1246.00	1816.00	1660.00	1040.00	323.00	198.00
Vehicle dispatching cost	300.00	110.00	80.00	40.00	150.00	30.00	480.00	520.00	400.00	170.00	170.00	40.00
Delivery cost	198.00	152.50	173.50	189.00	42.00	58.50	461.00	1373.00	1334.50	725.00	218.00	348.00
Penalty cost	45.00	30.00	175.00	160.00	0.00	20.00	350.00	595.00	935.00	365.00	240.00	110.00
Total cost	746.00	496.50	631.50	547.00	279.00	179.50	2537.00	4304.00	4329.50	2300.00	951.00	696.00
Total CPU time (s)	121.91	121.91	142.68	142.68	27.32	27.32	981.12	981.12	1420.88	1420.88	191.40	191.40
IRHA-II												
Number of stations	53	75	58	56	39	61	84	106	122	97	74	77
Number of vehicles	6	6	4	5	5	6	7	18	20	20	9	10
Number of served bikes	402	304	360	385	84	121	870	2596	2565	1452	445	710
UD	3	7	22	25	0	0	122	269	291	71	39	8
Vehicle traveling time cost	228.00	262.00	176.00	206.00	85.00	118.00	1024.00	1498.00	1248.00	1224.00	364.00	217.00
Vehicle dispatching cost	300.00	220.00	40.00	90.00	90.00	140.00	110.00	620.00	280.00	200.00	210.00	180.00
Delivery cost	201.00	152.00	180.00	192.50	42.00	60.50	435.00	1298.00	1282.50	726.00	222.50	355.00
Penalty cost	15.00	35.00	110.00	125.00	0.00	0.00	610.00	1345.00	1455.00	355.00	195.00	40.00
Total cost	744.00	669.00	506.00	613.50	217.00	318.50	2179.00	4761.00	4265.50	2505.00	991.50	792.00
Total CPU time (s)	259.68	400.88	342.54	198.98	29.85	449.70	1020.82	1508.06	1508.62	1509.92	176.81	176.81

rebalance to ensure an adequate supply of bikes at various stations to meet surging demands.

Secondly, in terms of practical decision-making, a cautious and conservative approach is recommended during off-peak periods. However, during peak demand periods, the platform should assign higher priority to potential future demands in their strategic planning.

Thirdly, the principle of “Take a step, take a look” with the RHA can effectively deal with the uncertain future demand and accelerate the decision-making process. Nevertheless, a predictive element should be integrated within the rolling horizon framework, transforming it from “Take a step, take a look” to “Take a step, take multiple looks”. This proactive strategy enables a better balance between operational costs and customer satisfaction by taking future demand projections into account.

In conclusion, managing BSS necessitates a nuanced interplay of responsive and anticipatory strategies. Managers must harness the insights gleaned from past trends, employ predictive analytics, and adopt flexible models to adeptly respond to the fluctuating patterns of user demand. Such a comprehensive approach ensures that the service not only addresses the current requirements of its clientele but is also well-positioned to adjust proactively to upcoming trends, ensuring the smooth operation of the system and its satisfaction in the minds of users.

6. Conclusions

In this study, we explore a dynamic bike repositioning problem under stochastic demand to address the spatiotemporal imbalance of bike-sharing systems caused by mismatched supply and demand. To account for both uncertainty and dynamic demand, a multi-period

mixed integer two-stage stochastic programming model is proposed to minimize the comprehensive costs. In the modeling process, the VNR strategy is incorporated by updating additional sets of departure and arrival stations to establish connections between vehicles’ routes across different periods. However, incorporating the VNR strategy introduces “loop by loop” decisions and presents challenges for multi-period optimization problems. While a traditional RHA can be utilized, but may suffer from poor decisions due to its myopic nature. Therefore, two improved rolling horizon approaches are developed transitioning a single-period rolling horizon to a r -period rolling horizon. Within each sub-period, the model is decomposed and solved by the hybrid parallel genetic algorithm and variable neighborhood search heuristic to enhance search ability and speed up the solving process.

To demonstrate the effectiveness of our proposed approach, computational tests are conducted from four perspectives. First, we calibrate the penalty coefficient parameter to ensure a high level of service quality. Then, the mathematical model is analyzed in terms of its scenario-based character and the incorporation of the VNR strategy. The scenario-based stochastic model enhances the robustness of dynamic repositioning across multiple scenarios, improving overall system stability and reducing the number of unmet demand by 95.48% on average, while the VNR strategy aligns better with practical applications, reducing operational costs by 29.04%. Next, the solution approach is evaluated in terms of the rolling horizon framework and heuristic algorithm. The results indicate that the rolling horizon framework enhances the performance of the repositioning scheme by effectively managing errors and optimizing solution accuracy and efficiency. Additionally, the hybrid PGA-VNS algorithm outperforms the Gurobi solver and other heuristics in terms of solution performance, with only

a minimal increase in computing time. The performance of the proposed method was ultimately validated through a real-world case study conducted in Beijing's Xicheng District, utilizing historical traveling data. The computational results provided compelling evidence that the proposed approach effectively tackles the challenges associated with spatiotemporal imbalances in bike flows. Additionally, it enhances the operational efficiency of the bike-sharing system while successfully meeting repositioning demands.

Future research should focus on the following aspects: (1) developing more refined models for addressing demand uncertainty in bike-sharing systems; (2) integrating demand forecasting algorithms with delivery decision algorithms to create a more realistic predictive-delivery model for bike-sharing systems; (3) exploring offline learning methods, such as reinforcement learning, to enhance the computational efficiency of our model in real-world applications.

CRediT authorship contribution statement

Xiang Li: Writing – review & editing, Supervision, Conceptualization. **Xianzhe Wang:** Writing – original draft, Visualization, Software. **Ziyan Feng:** Writing – review & editing, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. 71931001), Beijing Nova Program, China (No. 20230484421), the Fundamental Research Funds for the Central Universities, China (BH2322), and the Funds for First-class Discipline Construction, China (XK1802-5).

Appendix. Operators in our heuristic algorithm

A.1. Operators in GA

Genetic algorithms generally contain initializing population, calculating the fitness of chromosomes within the population, selection operator, crossover operator, and mutation operator, and we design each operator as follows:

1. Generate initial population: In genetic algorithms, each population consists of a series of chromosomes, and each chromosome corresponds to an initial feasible solution to the problem, using the initialization function, we generate chromosomes corresponding to the number of populations as the initial population.
2. Calculate fitness: The fitness of each chromosome is calculated by sorting the value of objective function within the population, obtaining the *index* of each chromosome after sorting, and calculating the fitness of each chromosome in terms of $fitness = 0.5 \cdot (1 - 0.5)^{index}$.
3. Selection operator: Selection operation refers selecting certain chromosomes according “survival fittest” rule. By using a roulette algorithm, good individuals are selected based on the fitness of each chromosome, while an elite strategy is applied to replace the worst chromosome in the population using the best chromosome in the population.

4. Crossover operator: The crossover operator allows the genes of good individuals in a population to be exchanged with each other in order to improve the performance of the population as a whole. Exchanging positions of stations of vehicle routes between different chromosomes is adopted to share good genes between chromosomes.
5. Mutation operator: The mutation operator introduces new and different solutions to the population by operating on individual chromosomes, and completes the mutation operation by randomly selecting a single route in a single period, fixing the departure and arrival stations, and randomly shuffling the stations between departure and arrival stations, or randomly generating a new solution to replace the original one using initialization function.

The designed crossover operators are introduced as follows.

1. Route Exchange Operator: Two chromosomes are randomly selected within each period. Randomly select two routes, fix the departure and arrival stations and exchange their remaining stations, and subsequently update the routes of other vehicles to form children chromosomes, see Fig. A.9(a).
2. Route Point Swap Operator: Two chromosomes are randomly selected within each period. A station is randomly selected in each chromosome, and the positions of randomly selected stations in the two routes are exchanged to form children chromosomes, see Fig. A.9(b).

The designed mutation operators are introduced as follows.

1. Point Exchange Operator: For a single chromosome, randomly select two routes. A station is randomly selected in each route, and the stations are then exchanged between the two routes to generate a new chromosome, see Fig. A.10(a).
2. Route Shift Operator: For a single chromosome, randomly select a route and randomly shuffle the stations between departure and arrival stations to generate a new chromosome, see Fig. A.10(b).

A.2. Operators in VNS

The designed ten neighborhood operators utilized to address our problem are introduced as follows.

1. Single-route two-point swap operator: An arbitrary route is randomly selected in each period and two stations that are not the departure and arrival stations of the route are randomly exchanged their positions in this route, as shown in Fig. A.11(a).
2. Single-route recombination operator: In a single route, fix the departure and arrival stations, and randomly shuffle the stations between departure and arrival stations, see Fig. A.11(b).
3. Single-route greedy recombination operator: In a single route, fix the departure and arrival stations, and rearrange that stations in a time-greedy method between departure and arrival stations, see Fig. A.11(c).
4. Stop at virtual station operator: In a single route returning to depot, force it to stop at virtual station, as shown in Fig. A.11(d).
5. Multi-routes two-point swap operator: Two routes are randomly selected within one period, and a station is randomly selected in each path, and the positions of randomly selected stations in the two routes are exchanged, as shown in Fig. A.11(e).
6. station shift operator: Two routes are randomly selected within one period, randomly select a single station in the first route and delete it and insert it into the second route in a shortest time-greedy method, see Fig. A.11(f).
7. Multi-routes all-point swap operator: Two routes are randomly selected within one period. Randomly select two routes, fix the

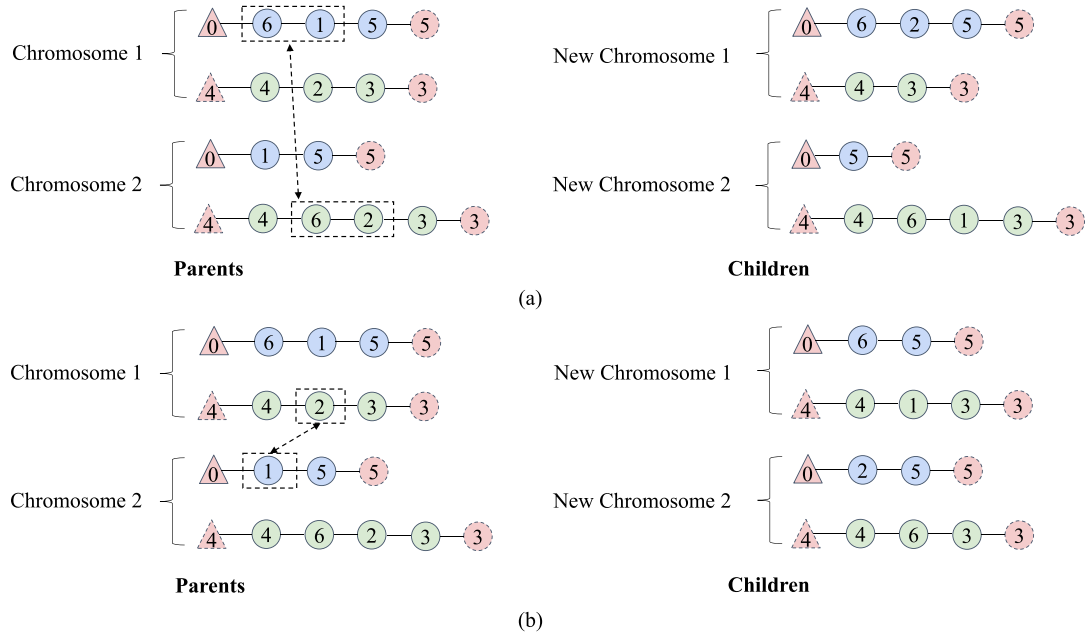


Fig. A.9. Crossover operators.

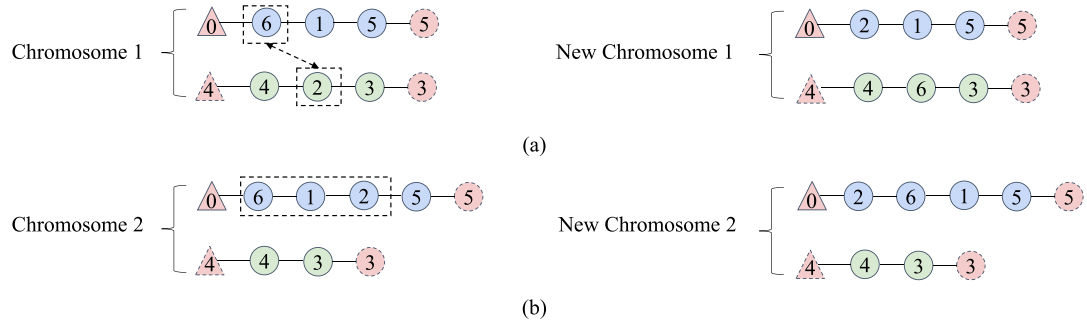


Fig. A.10. Mutation operators.

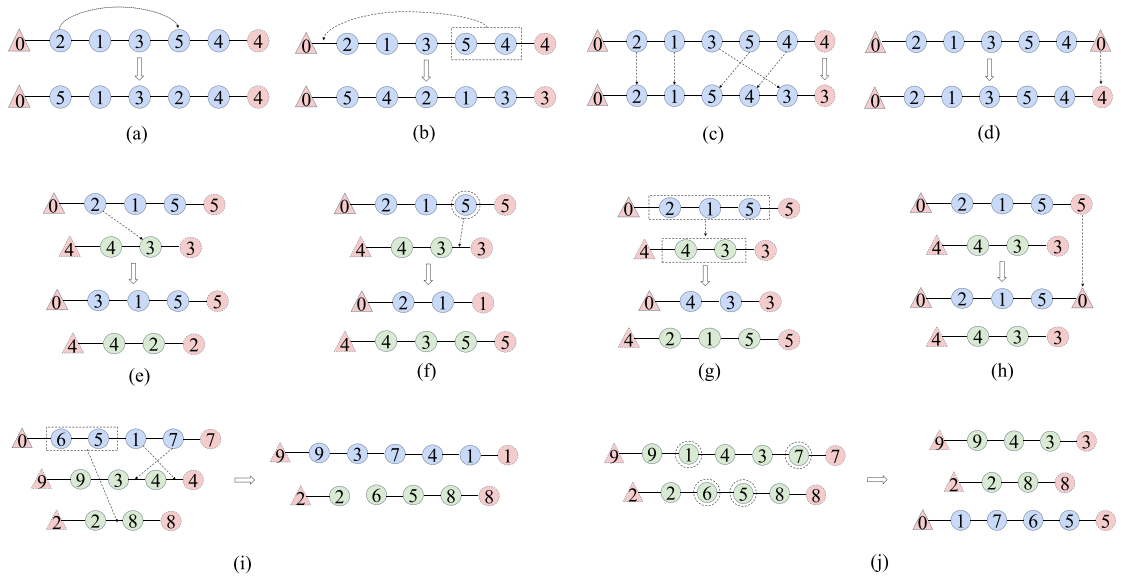


Fig. A.11. Perturbation and repair operators.

departure and arrival stations and swap their remaining stations, see Fig. A.11(g).

8. Return to depot operator: In a single route stopping virtual station, force it to return to depot., as shown in Fig. A.11(h).

9. Single-route deletion operator: Randomly select a route dispatched from the depot, scramble these stations and insert them into other routes in a time-greedy method, as depicted in Fig. A.11(i).
10. Single-route increase operator: Randomly select some routes dispatched from virtual stations. Some stations of these routes are randomly selected and scrambled to form a new route from the depot, as depicted in Fig. A.11(j).

References

- [1] Alvarez-Valdes R, Belenguer JM, Benavent E, Bermudez JD, Muñoz F, Vercher E, et al. Optimizing the level of service quality of a bike-sharing system. *Omega* 2016;62:163–75.
- [2] DeMaio P. Bike-sharing: History, impacts, models of provision, and future. *J Public Transport* 2009;12(4):41–56.
- [3] Kabra A, Belavina E, Girotra K. Bike-share systems: Accessibility and availability. *Manage Sci* 2020;66(9):3803–24.
- [4] Laporte G, Meunier F, Wolfier Calvo R. Shared mobility systems. *4or* 2015;13:341–60.
- [5] Wang Y, Yang Y, Wang J, Douglas M, Su D. Examining the influence of social norms on orderly parking behavior of dockless bike-sharing users. *Transp Res A* 2021;147:284–96.
- [6] De Chardon CM, Caruso G, Thomas I. Bike-share rebalancing strategies, patterns, and purpose. *J Transp Geogr* 2016;55:22–39.
- [7] Raviv T, Tzur M, Forma IA. Static repositioning in a bike-sharing system: Models and solution approaches. *EURO J Transp Logist* 2013;2(3):187–229.
- [8] Tan X, Zhu X, Li Q, Li L, Chen J. Tidal phenomenon of the dockless bike-sharing system and its causes: The case of Beijing. *Int J Sustain Transp* 2022;16(4):287–300.
- [9] Shui CS, Szeto W. Dynamic green bike repositioning problem—A hybrid rolling horizon artificial bee colony algorithm approach. *Transp Res D* 2018;60:119–36.
- [10] Fishman E. Bikeshare: A review of recent literature. *Transp Rev* 2016;36(1):92–113.
- [11] Illgen S, Höck M. Literature review of the vehicle relocation problem in one-way car sharing networks. *Transp Res B* 2019;120:193–204.
- [12] Shui C, Szeto W. A review of bicycle-sharing service planning problems. *Transp Res C* 2020;117:102648.
- [13] Angelopoulos A, Gavalas D, Konstantopoulos C, Kyriadias D, Pantziou G. Incentivized vehicle relocation in vehicle sharing systems. *Transp Res C* 2018;97:175–93.
- [14] Wu R, Liu S, Shi Z. Customer incentive rebalancing plan in free-float bike-sharing system with limited information. *Sustainability* 2019;11(11):3088.
- [15] Zhang J, Meng M, David Z. A dynamic pricing scheme with negative prices in dockless bike sharing systems. *Transp Res B* 2019;127:201–24.
- [16] Chen Q, Fu C, Zhu N, Ma S, He Q-C. A target-based optimization model for bike-sharing systems: From the perspective of service efficiency and equity. *Transp Res B* 2023;167:235–60.
- [17] Dell'Amico M, Iori M, Novellani S, Stützle T, et al. A destroy and repair algorithm for the bike sharing rebalancing problem. *Comput Oper Res* 2016;71:149–62.
- [18] Chemla D, Meunier F, Calvo RW. Bike sharing systems: Solving the static rebalancing problem. *Discrete Optim* 2013;10(2):120–46.
- [19] Liu Y, Szeto W, Ho SC. A static free-floating bike repositioning problem with multiple heterogeneous vehicles, multiple depots, and multiple visits. *Transp Res C* 2018;92:208–42.
- [20] Zhang B, Li X, Saldanha-da Gama F. Free-floating bike-sharing systems: New repositioning rules, optimization models and solution algorithms. *Inform Sci* 2022;600:239–62.
- [21] Caggiani L, Camporeale R, Ottomanelli M, Szeto WY. A modeling framework for the dynamic management of free-floating bike-sharing systems. *Transp Res C* 2018;87:159–82.
- [22] Du M, Cheng L, Li X, Tang F. Static rebalancing optimization with considering the collection of malfunctioning bikes in free-floating bike sharing system. *Transp Res E* 2020;141:102012.
- [23] Usama M, Shen Y, Zahoor O, Bao Q. Dockless bike-sharing system: Solving the problem of faulty bikes with simultaneous rebalancing operation. *J Transp Land Use* 2020;13(1):491–515.
- [24] Chang X, Wu J, Sun H, de Almeida Correia GH, Chen J. Relocating operational and damaged bikes in free-floating systems: A data-driven modeling framework for level of service enhancement. *Transp Res A* 2021;153:235–60.
- [25] Fu C, Zhu N, Ma S, Liu R. A two-stage robust approach to integrated station location and rebalancing vehicle service design in bike-sharing systems. *European J Oper Res* 2022;298(3):915–38.
- [26] Maggioni F, Cagnolari M, Bertazzi L, Wallace SW. Stochastic optimization models for a bike-sharing problem with transshipment. *European J Oper Res* 2019;276(1):272–83.
- [27] Dell'Amico M, Iori M, Novellani S, Subramanian A. The bike sharing rebalancing problem with stochastic demands. *Transp Res B* 2018;118:362–80.
- [28] Bashiri M, Nikzad E, Eberhard A, Hearne J, Oliveira F. A two stage stochastic programming for asset protection routing and a solution algorithm based on the progressive hedging algorithm. *Omega* 2021;104:102480.
- [29] Legros B. Dynamic repositioning strategy in a bike-sharing system; how to prioritize and how to rebalance a bike station. *European J Oper Res* 2019;272(2):740–53.
- [30] Luo X, Li L, Zhao L, Lin J. Dynamic intra-cell repositioning in free-floating bike-sharing systems using approximate dynamic programming. *Transp Sci* 2022;56(4):799–826.
- [31] Zhang D, Yu C, Desai J, Lau H, Srivathsan S. A time-space network flow approach to dynamic repositioning in bicycle sharing systems. *Transp Res B* 2017;103:188–207.
- [32] Erdoğan G, Battarra M, Calvo RW. An exact algorithm for the static rebalancing problem arising in bicycle sharing systems. *European J Oper Res* 2015;245(3):667–79.
- [33] Al Chami Z, Manier H, Manier M-A. Multi-period pickup and delivery problem with time windows and paired demands. In: 2018 15th international conference on control, automation, robotics and vision. IEEE; 2018, p. 337–42.
- [34] Ho SC, Szeto WY. A hybrid large neighborhood search for the static multi-vehicle bike-repositioning problem. *Transp Res B* 2017;95:340–63.
- [35] Zhan S, Kroon LG, Zhao J, Peng Q. A rolling horizon approach to the high speed train rescheduling problem in case of a partial segment blockage. *Transp Res E* 2016;95:32–61.
- [36] Wu O, Dalle Ave G, Harjunkoski I, Imsland L. A rolling horizon approach for scheduling of multiproduct batch production and maintenance using generalized disjunctive programming models. *Comput Chem Eng* 2021;148:107268.
- [37] Fattahi M, Govindan K. Data-driven rolling horizon approach for dynamic design of supply chain distribution networks under disruption and demand uncertainty. *Decis Sci* 2022;53(1):150–80.
- [38] Glomb L, Liers F, Rösel F. A rolling-horizon approach for multi-period optimization. *European J Oper Res* 2022;300(1):189–206.
- [39] Dib O, Manier M-A, Moalic L, Caminada A. Combining VNS with genetic algorithm to solve the one-to-one routing issue in road networks. *Comput Oper Res* 2017;78:420–30.
- [40] Liu M, Liu Z, Chu F, Dolgui A, Chu C, Zheng F. An optimization approach for multi-echelon supply chain viability with disruption risk minimization. *Omega* 2022;112:102683.
- [41] Pei J, Liu X, Fan W, Pardalos PM, Lu S. A hybrid BA-VNS algorithm for coordinated serial-batching scheduling with deteriorating jobs, financial budget, and resource constraint in multiple manufacturers. *Omega* 2019;82:55–69.
- [42] Worasan K, Sethanan K, Pitakaso R, Jamrus T, Moonsri K, Golinska-Dawson P. A hybridization of PSO and VNS to solve the machinery allocation and scheduling problem under a machinery sharing arrangement. *Intell Syst Appl* 2023;18:200206.